

# Transmission vidéo basse latence pour drone FPV

Travail de Bachelor, rendu intermédiaire

Non Confidentiel

Départements : TIC

Filière : Informatique et systèmes de communication

Orientation : Systèmes informatiques embarqués

**Auteur :** Tschantz Nathan

**Supervisé par :** Favrat Pierre

**Date :** 17 mai 2026



## 1 Préambule

Ce travail de Bachelor (ci-après **TB**) est réalisé en fin de cursus d'études, en vue de l'obtention du titre de Bachelor of Science HES-SO en Ingénierie.

En tant que travail académique, son contenu, sans préjuger de sa valeur, n'engage ni la responsabilité de l'auteur, ni celles du jury du travail de Bachelor et de l'École.

Toute utilisation, même partielle, de ce TB doit être faite dans le respect du droit d'auteur.

HEIG-VD  
Le Chef du Département

Yverdon-les-Bains, le 17 mai 2026



## 2 Authentification

Je soussigné, Nathan Tschantz, atteste par la présente avoir réalisé seul ce travail et n'avoir utilisé aucune autre source que celles expressément mentionnées.

Nathan Tschantz



Yverdon-les-Bains, le 17 mai 2026



### 3 Résumé - français

**Contexte :** Le pilotage de drones FPV en environnements complexes nécessite un retour vidéo en temps réel. Or, les fréquences standards (2.4 et 5.8 GHz) pénètrent mal les obstacles physiques (conditions NLOS).

**Problématique :** La norme Wi-Fi HaLow (IEEE 802.11ah, Sub-1 GHz) offre une excellente pénétration, mais ses mécanismes natifs de fiabilité (acquittements MAC, retransmissions) induisent une latence et une gigue incompatibles avec les exigences strictes du vol en immersion.

**Méthode :** Ce travail implémente une architecture de bout en bout forçant une transmission unidirectionnelle. Un pont matériel (SPI vers microcontrôleur STM32) encapsule un flux vidéo H.264 (optimisé via GStreamer) en UDP Broadcast sans accusé de réception, verrouillant le modem radio sur une modulation robuste.

**Résultat intermédiaire :** Les mesures démontrent une latence de transport radio pur d'environ 12.5 ms. La latence visuelle globale (*Glass-to-Glass*) se situe actuellement entre 130 et 140 ms. La suite du projet visera à trouver la source de ce délai résiduel et à tenter de l'éliminer.

### 4 Abstract - English

**Context :** FPV drone piloting in complex environments requires a real-time video feed. However, standard frequencies (2.4 and 5.8 GHz) struggle to penetrate physical obstacles (NLOS conditions).

**Problem :** The Wi-Fi HaLow standard (IEEE 802.11ah, Sub-1 GHz) offers excellent obstacle penetration, but its native reliability mechanisms (MAC acknowledgments, retransmissions) introduce latency and jitter incompatible with the strict requirements of immersive flight.

**Method :** This work implements an end-to-end architecture forcing unidirectional transmission. A hardware bridge (SPI to STM32 microcontroller) encapsulates an optimized H.264 video stream (via GStreamer) into UDP Broadcast with no acknowledgments, locking the radio modem onto a robust modulation scheme.

**Intermediate Result :** Profiling demonstrates a pure radio transport latency of approximately 12.5 ms. The overall *Glass-to-Glass* visual latency is currently between 130 and 140 ms. The next phase of the project will focus on understanding and reducing this residual delay.





## Table des matières

|           |                                                                       |           |
|-----------|-----------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Préambule</b>                                                      | <b>3</b>  |
| <b>2</b>  | <b>Authentification</b>                                               | <b>5</b>  |
| <b>3</b>  | <b>Résumé - français</b>                                              | <b>7</b>  |
| <b>4</b>  | <b>Abstract - English</b>                                             | <b>7</b>  |
| <b>5</b>  | <b>Terminologie et concepts de base</b>                               | <b>14</b> |
|           | <b>Terminologie et concepts clés</b>                                  | <b>14</b> |
| <b>6</b>  | <b>Introduction</b>                                                   | <b>16</b> |
| 6.1       | Contexte . . . . .                                                    | 16        |
| 6.2       | Problématique . . . . .                                               | 16        |
| 6.3       | Objectifs du travail de Bachelor . . . . .                            | 16        |
| 6.3.1     | Définition du seuil d'acceptabilité et cas d'usage . . . . .          | 17        |
| 6.4       | Structure du document . . . . .                                       | 17        |
| <b>7</b>  | <b>Étude de faisabilité</b>                                           | <b>18</b> |
| 7.1       | But . . . . .                                                         | 18        |
| 7.2       | Méthodologie d'investigation . . . . .                                | 18        |
| 7.3       | Faisabilité côté Émetteur (FreeRTOS / STM32) . . . . .                | 18        |
| 7.4       | Faisabilité côté Récepteur (Linux / OpenWrt) . . . . .                | 18        |
| 7.5       | Conclusion de la faisabilité . . . . .                                | 19        |
| <b>8</b>  | <b>Architecture globale du système</b>                                | <b>20</b> |
| 8.1       | Le module d'acquisition et d'émission embarqué (TX) . . . . .         | 20        |
| 8.2       | Le pont réseau de réception au sol (routeur RX) . . . . .             | 21        |
| 8.3       | La station de décodage et d'affichage . . . . .                       | 21        |
| 8.4       | Précision de la structure du document . . . . .                       | 22        |
| <b>9</b>  | <b>Configuration de l'environnement de développement embarqué</b>     | <b>23</b> |
| 9.1       | Intégration du SDK sous Windows . . . . .                             | 23        |
| 9.2       | Adaptation de la chaîne de cross compilation . . . . .                | 23        |
| 9.3       | Configuration du débogage matériel (ST-LINK) . . . . .                | 24        |
| 9.4       | Provisionnement et calibration radio (Config Store) . . . . .         | 24        |
| 9.4.1     | Architecture de flashage . . . . .                                    | 24        |
| 9.4.2     | Résolution du conflit de signature et de version matérielle . . . . . | 24        |
| <b>10</b> | <b>Implémentation du réseau Wi-Fi HaLow</b>                           | <b>26</b> |
| 10.1      | Stratégie de diffusion (UDP Broadcast et No-ACK) . . . . .            | 26        |
| 10.2      | Optimisation de la réactivité radio : Désactivation du PSM . . . . .  | 26        |
| 10.3      | Contournement des contraintes réglementaires (ETSI vs FCC) . . . . .  | 27        |
| 10.4      | Le routage et l'isolation IP (OpenWrt) . . . . .                      | 27        |

|           |                                                                           |           |
|-----------|---------------------------------------------------------------------------|-----------|
| 10.5      | Validation empirique et mathématique de la bande passante vidéo . . . . . | 28        |
| 10.5.1    | Impact de la compression sur la fragmentation des paquets . . . . .       | 29        |
| <b>11</b> | <b>Capture et traitement du flux vidéo</b>                                | <b>30</b> |
| 11.1      | Arbitrage entre les normes H.264 et H.265 . . . . .                       | 30        |
| 11.2      | Le framework GStreamer et l'optimisation H.264 . . . . .                  | 30        |
| 11.2.1    | Précisions sur les paramètres de pipeline . . . . .                       | 30        |
| 11.3      | Pipelines d'émission (TX) . . . . .                                       | 31        |
| 11.4      | Pipelines de réception et d'affichage (RX) . . . . .                      | 31        |
| 11.5      | Identification et contournement de la latence de l'ISP . . . . .          | 32        |
| 11.6      | Outils de diagnostic réseau . . . . .                                     | 32        |
| <b>12</b> | <b>Interfaçage matériel et transfert SPI</b>                              | <b>33</b> |
| 12.1      | Identification du bus SPI matériel . . . . .                              | 33        |
| 12.2      | Portage de l'application d'émission (De Python à C) . . . . .             | 33        |
| 12.3      | Contrôle de flux matériel (Handshake SPI) . . . . .                       | 34        |
| <b>13</b> | <b>Itérations matérielles et analyse des performances</b>                 | <b>35</b> |
| 13.1      | Profilage théorique de la chaîne d'émission . . . . .                     | 35        |
| 13.1.1    | Temps d'encodage matériel (Jetson Nano) . . . . .                         | 35        |
| 13.1.2    | Mesure au cycle d'horloge du traitement MCU (STM32) . . . . .             | 35        |
| 13.1.3    | Bilan de la latence d'émission par image . . . . .                        | 35        |
| 13.2      | Caractérisation physique du transport radio . . . . .                     | 36        |
| 13.3      | Isolement de la capture et abandon de la piste OpenHD . . . . .           | 36        |
| 13.4      | Bilan "Glass-to-Glass" et latence résiduelle . . . . .                    | 37        |
| 13.5      | Bilan récapitulatif des temps de traitement . . . . .                     | 37        |
| 13.6      | Bilan "Glass-to-Glass" et perspectives (Rendu intermédiaire) . . . . .    | 38        |
| <b>14</b> | <b>Conclusion intermédiaire</b>                                           | <b>40</b> |
| 14.1      | Bilan des réalisations . . . . .                                          | 40        |
| 14.2      | Objectifs pour la phase finale . . . . .                                  | 41        |
| <b>15</b> | <b>Note sur l'utilisation de l'intelligence artificielle</b>              | <b>42</b> |
| <b>16</b> | <b>Bibliographie</b>                                                      | <b>43</b> |
| <b>A</b>  | <b>Guide de connexion et d'accès aux équipements</b>                      | <b>44</b> |
| A.1       | Accès au microcontrôleur d'émission (STM32 / EKH05) . . . . .             | 44        |
| A.2       | Accès aux ordinateurs de capture (Jetson Nano / RPi 4B) . . . . .         | 44        |
| A.3       | Accès au routeur relais de réception (GL.iNet MT3000 / OpenWrt) . . . . . | 44        |
| A.4       | Accès et adressage des stations de réception (RX) . . . . .               | 45        |
| A.5       | Tableau récapitulatif d'adressage et d'accès . . . . .                    | 45        |
| <b>B</b>  | <b>Modifications du pilote Morse Micro</b>                                | <b>46</b> |

---

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| <b>C</b> | <b>Mesure de latence via bouton partagé</b>                    | <b>46</b> |
| C.1      | Le Code TX (Émetteur de test : latency_tx.c) . . . . .         | 46        |
| C.2      | Le Code RX (Chronomètre de précision : latency_rx.c) . . . . . | 48        |
| <b>D</b> | <b>Code TX</b>                                                 | <b>51</b> |
| D.1      | JETSON (jetson_to_rpi.c) . . . . .                             | 51        |
| D.2      | RPI4B (rpi4b_to_rpi.c) . . . . .                               | 54        |
| <b>E</b> | <b>Firmware MCU (spi_interface.c)</b>                          | <b>57</b> |
|          | <b>Glossaire</b>                                               | <b>67</b> |
|          | <b>Index</b>                                                   | <b>69</b> |

## Table des figures

|   |                                                                                                                                                 |    |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1 | structure globale du système . . . . .                                                                                                          | 17 |
| 2 | Architecture globale précisée . . . . .                                                                                                         | 20 |
| 3 | Flux logique du système : (1) Lien réseau Sub-1 GHz, (2) Capture vidéo,<br>(3) Interfaçage SPI, (4) Plateformes de traitement TX et RX. . . . . | 22 |
| 4 | Configuration du serveur GDB ST-LINK dans STM32CubeIDE . . . . .                                                                                | 24 |
| 5 | Schéma du système de test . . . . .                                                                                                             | 36 |

## Liste des tableaux

|   |                                                                                 |    |
|---|---------------------------------------------------------------------------------|----|
| 1 | Routage du bus SPI sur la carte d'évaluation EKH05 . . . . .                    | 33 |
| 2 | Récapitulatif des accès et du plan d'adressage (Sous-réseau FPV : 192.168.12.x) | 45 |

## 5 Terminologie et concepts de base

Afin de faciliter la lecture de ce document et la compréhension des enjeux techniques liés à la transmission vidéo basse latence, les concepts et acronymes suivants seront couramment utilisés :

**Wi-Fi HaLow (IEEE 802.11ah)** : Amendement du standard Wi-Fi conçu pour opérer dans les bandes de fréquences inférieures à 1 GHz (Sub-1 GHz). Contrairement au Wi-Fi classique (2.4 GHz ou 5.8 GHz), il privilégie la portée (jusqu'à plusieurs kilomètres) et la capacité à traverser les obstacles physiques, au détriment de la bande passante maximale.

**FPV (*First Person View*)** : "Vol en immersion". Technique de pilotage où la caméra embarquée sur le drone transmet un flux vidéo en temps réel vers les lunettes ou l'écran du pilote, exigeant une latence extrêmement faible et constante.

**NLOS (*Non-Line-Of-Sight*)** : Situation de communication radio où l'émetteur et le récepteur ne sont pas en ligne de vue directe (présence de bâtiments, de végétation ou de relief). C'est dans ces conditions que les hautes fréquences perdent leur efficacité.

**MCS (*Modulation and Coding Scheme*)** : Indice (généralement de 0 à 10) définissant le débit physique de la transmission. Il combine un type de modulation radio (ex : BPSK, QAM) et un taux de codage. Un MCS bas (ex : 1 ou 2) offre un débit réduit, mais garantit une robustesse maximale face au bruit et aux interférences.

**LDPC (*Low-Density Parity-Check*)** : Algorithme avancé de correction d'erreurs (FEC). Contrairement au codage convolutif standard (BCC), le LDPC utilise des matrices de parité complexes permettant au récepteur de reconstruire des paquets de données corrompus par de mauvaises conditions radio, évitant ainsi de devoir redemander l'envoi du paquet (ce qui détruirait la latence vidéo).

**Couches PHY et MAC** : Niveaux fondamentaux du modèle réseau. La **couche PHY** (Physique) gère la transformation des bits en ondes radio. La **couche MAC** (*Media Access Control*) gère les règles d'accès à l'antenne et les retransmissions. Ce projet modifie directement ces couches pour les adapter au flux vidéo.

**RTOS (*Real-Time Operating System*)** : Système d'exploitation temps réel (comme FreeRTOS, utilisé sur le microcontrôleur d'émission STM32). Contrairement à un OS classique (comme Linux), un RTOS garantit l'exécution des tâches dans des délais stricts et déterministes, minimisant ainsi la gigue logicielle lors du routage des données.

**Latence "Glass-to-Glass"** : Délai total mesuré entre l'instant où un événement physique se produit devant l'objectif de la caméra (le premier "verre") et le moment où il est affiché sur l'écran du pilote (le second "verre"). C'est la métrique de performance absolue de ce projet, englobant tous les temps de traitement matériels et logiciels.

**UDP Broadcast & No-ACK** : Stratégie de transmission réseau consistant à diffuser des paquets de données à l'aveugle à tous les équipements du sous-réseau, sans demander d'accusé de réception (*No-ACK*). Cette méthode sacrifie la garantie absolue de livraison au profit d'un flux continu à latence minimale.

**Gigue (*Jitter*) :** Variation temporelle de la latence. Dans un flux vidéo en temps réel, une forte gigue (des paquets arrivant en rafales irrégulières) provoque des saccades visuelles souvent plus perturbantes pour le pilotage qu'une latence fixe et stable.

## 6 Introduction

### 6.1 Contexte

Le pilotage de drones FPV (*First Person View*) repose sur un retour vidéo en temps réel. Pour garantir la sécurité du vol et la réactivité du pilote, la fiabilité de la liaison radio et la minimisation de la latence sont absolument critiques. Actuellement, la grande majorité des retours vidéo commerciaux reposent sur des liaisons fonctionnant dans les bandes de fréquences de 2.4 GHz ou 5.8 GHz. Bien que ces hautes fréquences offrent une large bande passante permettant le transfert de vidéos en haute définition, leur capacité de pénétration est en revanche médiocre face aux obstacles physiques.

En environnement urbain, forestier ou industriel (conditions dites *Non-Line-Of-Sight* ou NLOS), la portée de ces systèmes vidéo se trouve drastiquement réduite. À l'inverse, les protocoles opérant dans les bandes inférieures à 1 GHz (Sub-1 GHz, tels que TBS Crossfire ou ExpressLRS) sont devenus le standard industriel pour le lien de contrôle (la radiocommande) grâce à leur robustesse face aux obstacles, mais leur bande passante a toujours été techniquement insuffisante pour y faire transiter de la vidéo.

### 6.2 Problématique

Pour pallier ce manque et réunir le meilleur des deux mondes, la norme IEEE 802.11ah, également connue sous le nom de Wi-Fi HaLow, se présente comme une alternative prometteuse. Opérant dans la bande de fréquences Sub-1 GHz, elle offre une portée kilométrique et une excellente capacité à traverser les matériaux, tout en proposant des débits de l'ordre du Mégabit par seconde.

Cependant, les standards Wi-Fi classiques n'ont pas été conçus à l'origine pour transporter des flux vidéo continus et déterministes à très faible latence. Les mécanismes natifs d'adaptation de débit (*Rate Control*) et les retransmissions automatiques de paquets visent à garantir l'intégrité absolue des données au détriment du temps de livraison. Pour le vol FPV, l'attente d'une retransmission introduit une gigue (*jitter*) et une latence totalement incompatibles avec les exigences de pilotage.

### 6.3 Objectifs du travail de Bachelor

L'objectif principal de ce projet est d'étudier, de concevoir et de valider de bout en bout un prototype de transmission vidéo exploitant les avantages physiques de la norme IEEE 802.11ah (Wi-Fi HaLow).

Plutôt que de se limiter à la simple configuration logicielle de modules radio, l'enjeu s'est révélé être une véritable optimisation du système dans son ensemble. Cela implique le contournement des mécanismes d'acquiescement réseau (diffusion unidirectionnelle), le réglage strict des pipelines de compression vidéo matérielle, ainsi que la mise en place d'une synchronisation physique entre les différents processeurs et microcontrôleurs embarqués pour éviter l'accumulation de données dans les mémoires tampons (*buffers*).



### 6.3.1 Définition du seuil d'acceptabilité et cas d'usage

Au-delà de l'optimisation technique de la chaîne, il convient de définir un objectif de latence global pragmatique, aligné avec les contraintes physiques du vol en immersion. Sur une machine de type "freestyle" ou de course, les systèmes de transmission numériques traditionnels (opérant en 5.8 GHz) ciblent une latence de 20 à 30 ms (voire inférieure) pour autoriser des manœuvres très agressives.

Toutefois, l'objectif de ce travail n'est pas de concurrencer ces systèmes sur la vitesse pure, mais de privilégier la robustesse et la pénétration de signal en milieu complexe (conditions NLOS telles que l'exploration de bâtiments isolés, de forêts denses ou de tunnels). Dans ce contexte d'exploration, une absence de perte de liaison est préférable à une latence absolue minimale.

Le pilotage reste parfaitement viable et sécurisant jusqu'à un seuil de latence d'environ 80 ms. L'objectif de ce système, une fois toutes les optimisations matérielles et logicielles appliquées, est donc d'atteindre ce seuil des 80 ms, qui représente une limite physique raisonnable au vu de la chaîne d'acquisition matérielle grand public exploitée dans ce projet.

## 6.4 Structure du document

Ce rapport s'articule autour d'une progression logique, allant de la mise en place des fondations matérielles et logicielles jusqu'à l'optimisation itérative du système complet. Après une étude de faisabilité validant nos choix technologiques, nous détaillerons l'architecture globale retenue.



FIGURE 1 – structure globale du système

Dans un premier temps, nous aborderons la configuration de l'environnement de développement et l'implémentation spécifique du lien réseau Wi-Fi HaLow, éléments qui constituent le socle de notre système.

Une fois ce pont réseau établi, nous décrirons les mécanismes logiciels de capture et de traitement de la vidéo, puis nous détaillerons l'interfaçage matériel (via le bus SPI) permettant d'acheminer ce flux du processeur vers le microcontrôleur d'émission.

Enfin, la dernière partie de ce rapport sera consacrée à l'approche itérative du projet : où seront présentées les différentes combinaisons de plateformes matérielles évaluées (Nvidia Jetson Nano, Raspberry Pi, PC) et où sera exposée l'analyse des mesures de latence ayant permis d'isoler les différents goulots d'étranglement de la chaîne.

## 7 Étude de faisabilité

### 7.1 But

Pour réaliser une transmission vidéo robuste face aux obstacles, notre stratégie repose sur le forçage d'un MCS bas et l'utilisation de la technique de correction d'erreurs LDPC. L'objectif de cette étude de faisabilité est de déterminer si ces paramètres, généralement gérés dynamiquement par le matériel, sont configurables manuellement sur les modules Wi-Fi HaLow Morse Micro MM8108 utilisés pour ce projet.

### 7.2 Méthodologie d'investigation

Dans un premier temps, l'analyse de la documentation constructeur et des interfaces utilisateur (*user-friendly*) d'OpenWrt a révélé que si certains paramètres basiques sont accessibles (comme le choix du canal ou de la largeur de bande), l'activation forcée du LDPC ou le verrouillage strict du MCS ne sont pas exposés par les utilitaires standards (tels que `iw`, `morse_cli` ou `hostapd`). L'investigation s'est donc orientée vers le reverse engineering et l'analyse du code source des pilotes fournis par Morse Micro, tant pour l'environnement FreeRTOS (Émetteur embarqué) que pour l'environnement Linux (Routeur relais).

### 7.3 Faisabilité côté Émetteur (FreeRTOS / STM32)

L'émetteur (TX), chargé de récupérer la vidéo et d'émettre les ondes radio depuis le drone, repose sur un microcontrôleur STM32 exploitant le système d'exploitation temps réel FreeRTOS. L'investigation du kit de développement logiciel (`mm-iot-sdk`) a permis d'identifier les leviers d'action suivants :

- **Contrôle du MCS** : L'analyse des bibliothèques a mis en évidence la fonction interne `mmwlan_ate_override_rate_control()` issue du fichier d'en-tête `mmwlan.h`. Cette fonction de test (ATE) permet de verrouiller dynamiquement la bande passante et le MCS directement dans le code source de l'application en C.
- **Activation du LDPC** : L'inspection des configurations internes, notamment celles liées au démon `wpa_supplicant` (`config_ssid.h`), indique que la fonctionnalité LDPC est activée par défaut par le pilote lors de la négociation de la connexion.

### 7.4 Faisabilité côté Récepteur (Linux / OpenWrt)

Le relais radio, configuré en point d'accès (Access Point, AP) au sol pour réceptionner le flux, repose sur une architecture Linux (OpenWrt). L'analyse du code source du pilote noyau (*kernel driver*) a révélé les éléments suivants :

- **Contrôle du MCS** : Les fichiers sources `rc.c` et `command.c` contiennent des structures permettant de désactiver l'algorithme d'adaptation de débit (`enable_fixed_rate`) et d'imposer un MCS fixe (par exemple, MCS 2). Des tests de compilation croisée (*cross-compilation*) du noyau ont confirmé que ces modifications logicielles peuvent être compilées sous forme de module (`morse.ko`) et injectées à chaud sur le routeur cible.

- **La limite du forçage LDPC** : Initialement, une modification directe des en-têtes de paquets MAC a été explorée. L'étude du fichier `skbq.c` laissait supposer la possibilité d'intercepter les trames pour manipuler le registre `morse_ratecode` (en forçant les bits B17 et B18 du champ SIG-1). Cependant, les expérimentations ont démontré qu'il est techniquement impossible d'appliquer et de maintenir ces paramètres matériels bas niveau tant que le routeur n'est pas formellement connecté à un client.

La preuve formelle d'une injection manuelle du LDPC n'a pas pu être clairement établie et a donc été écartée de l'implémentation finale. Néanmoins, la présence active des paramètres liés au LDPC et au MCS 10 dans le code source du pilote confirme la capacité de la puce à utiliser cette correction d'erreur en interne. Nous partons donc du postulat techniquement fondé que le contrôleur radio l'exploite de manière native.

## 7.5 Conclusion de la faisabilité

L'étude démontre que l'objectif de manipulation de la couche physique est réalisable sur les points critiques. S'il s'est avéré impraticable de forger manuellement chaque en-tête MAC pour imposer le LDPC en raison de la machine d'états du standard Wi-Fi, l'accès au code source du SDK FreeRTOS et des pilotes Linux permet d'appliquer les surcharges (*overrides*) nécessaires pour verrouiller l'utilisation d'un MCS bas. Ces découvertes valident la faisabilité d'une liaison radio déterministe, prérequis absolu pour notre système vidéo.

## 8 Architecture globale du système

Afin de répondre aux exigences de la transmission vidéo basse latence pour le pilotage FPV, l'architecture du système a été pensée de manière modulaire. La chaîne de transmission a été segmentée en trois entités fonctionnelles et matérielles distinctes :

1. **Le module d'acquisition et d'émission embarqué (TX)** : Destiné à être monté sur le drone, il est chargé de la capture vidéo brute, de sa compression, et de son émission radio sur la bande Sub-1 GHz.
2. **Le pont réseau de réception au sol (routeur RX)** : Il agit comme une passerelle réseau transparente. Son unique rôle est de réceptionner les ondes radio HaLow et de les convertir en trames Ethernet physiques, sans effectuer de traitement logiciel.
3. **La station de décodage et d'affichage** : Connectée au pont réseau via un câble Ethernet, cette entité est exclusivement dédiée au traitement logiciel du flux vidéo entrant pour l'afficher sur l'écran du pilote avec le délai le plus court possible.

Cette segmentation architecturale offre un double avantage : elle permet de décharger les microcontrôleurs Wi-Fi des lourdes tâches d'encodage/décodage vidéo, et elle a rendu possible le profilage de chaque sous-système séparément tout au long de la phase de développement.

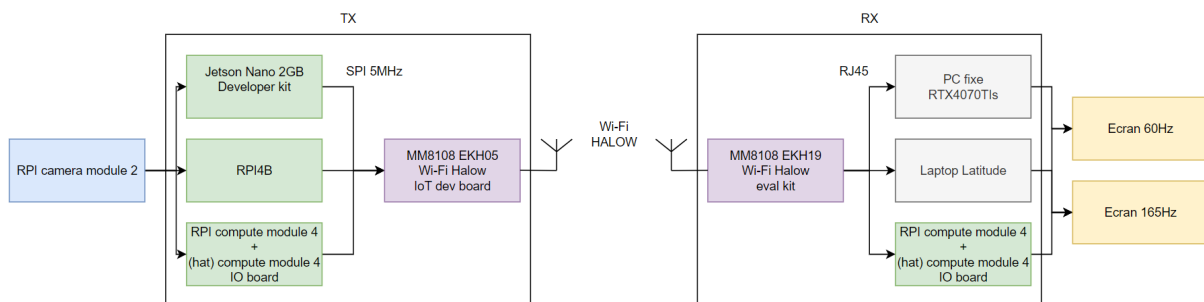


FIGURE 2 – Architecture globale précisée

### 8.1 Le module d'acquisition et d'émission embarqué (TX)

L'émetteur embarqué constitue le point de départ de la chaîne d'acquisition. Le module de traitement vidéo n'a pas été figé immédiatement ; plusieurs ordinateurs ont été évalués successivement pour isoler les variables de performance :

- Une **Nvidia Jetson Nano 2GB**, utilisée initialement pour exploiter son encodeur matériel intégré (NVENC) sur le flux brut du capteur CMOS. Le choix de cette plateforme particulièrement puissante visait à éliminer toute inconnue quant aux capacités d'encodage. En établissant une base de référence fiable avec un matériel surdimensionné, il a été possible de valider le pont de communication SPI et la liaison radio sans douter des performances de la source.

- Des **Raspberry Pi 4** (une *Compute Module 4 IO Board* puis une *Raspberry Pi 4B* classique), testées dans un second temps pour évaluer des pipelines de capture vidéo alternatifs dans une démarche de *downscaling*, une fois l'architecture réseau validée. De plus cela a permis d'utiliser les bibliothèques Raspberry natives à la caméra (raspberrypi camera module 2).

Indépendamment de l'ordinateur utilisé pour la compression (H.264 / H.265), les trames sont ensuite systématiquement transmises via un bus SPI cadencé à 5 MHz vers un micro-contrôleur **STM32U5**. Ce MCU agit comme un pont réseau déterministe à ultra-haute vitesse. Il encapsule les données vidéo brutes dans des paquets UDP à l'aide de la pile réseau LwIP, avant de les expédier au module radio **Morse Micro MM8108 (EKH05)** pour la transmission RF sur la bande Sub-1 GHz.

## 8.2 Le pont réseau de réception au sol (routeur RX)

Contrairement aux systèmes Wi-Fi traditionnels où le point d'accès traite intensivement les paquets, le relais radio de notre architecture est conçu pour être le plus transparent possible et est resté constant durant tout le projet.

La réception des ondes HaLow est assurée par un routeur **GL.iNet MT3000** équipé d'une carte d'évaluation **EKH19**. Ce routeur, fonctionnant sous le système d'exploitation OpenWrt, est configuré pour agir comme un simple pont logiciel (*Bridge*). Il réceptionne les trames 802.11ah sur son interface sans-fil et les injecte directement sur son interface Ethernet physique, limitant ainsi le temps de traitement logiciel par le CPU du routeur.

## 8.3 La station de décodage et d'affichage

La station au sol est le dernier maillon de la chaîne, responsable du décodage et de l'affichage. L'objectif de ce projet étant d'observer et de maîtriser chaque source de latence, deux environnements radicalement différents ont été employés :

- **La station de diagnostic (PC Windows)** : Son utilisation a été temporaire et diagnostique. Afin de maintenir un environnement d'analyse logiciel constant, la réception et le décodage ont été réalisés via le framework **GStreamer**. Des tests croisés ont été menés entre un ordinateur portable (Dell Latitude, écran 60 Hz et 165Hz) et une station fixe (NVIDIA RTX 4070 Ti Super, écran 165 Hz). L'unique but de cette plateforme était de prouver que le temps de traitement brut du processeur n'était pas le goulot d'étranglement de l'architecture. Une fois cette hypothèse écartée, l'environnement Windows a été abandonné au profit de Linux. Il est toutefois à noter que sur le plan de la latence, les mesures de latence "Glass-to-Glass" les plus faibles relevées sur cette plateforme ont logiquement été observées en utilisant l'écran 165 Hz.
- **Les stations cibles (Raspberry Pi 4 - CM4 et 4B)** : Le passage définitif sur ces plateformes ARM s'inscrit dans une volonté d'utiliser un système d'exploitation Linux, offrant un meilleur contrôle sur le matériel. L'objectif était de s'affranchir des limitations et de l'overhead de windows. Ces plateformes ont permis d'évaluer et d'implémenter des pipelines logiciels GStreamer également.

## 8.4 Précision de la structure du document

Comme déterminé précédemment, voici l'ordre dans lequel ce rapport sera structuré de manière plus détaillé.

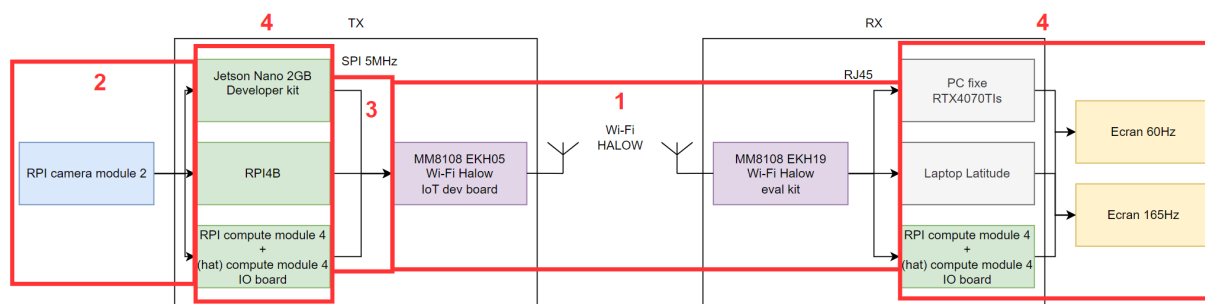


FIGURE 3 – Flux logique du système : (1) Lien réseau Sub-1 GHz, (2) Capture vidéo, (3) Interfaçage SPI, (4) Plateformes de traitement TX et RX.

## 9 Configuration de l'environnement de développement embarqué

Le développement du firmware pour le microcontrôleur gérant la transmission (STM32) repose sur le kit de développement logiciel officiel de Morse Micro (`mm-iot-sdk`). Ce SDK étant nativement conçu pour un environnement Linux avec platformio et des chaînes de cross compilation préinstallées, son intégration sous Windows a nécessité plusieurs adaptations.

### 9.1 Intégration du SDK sous Windows

Le code source a été récupéré via le dépôt GitHub officiel. L'architecture du projet s'appuyant sur plusieurs dépendances externes vitales (telles que le système d'exploitation temps réel FreeRTOS, la pile réseau LwIP et mbedTLS), une initialisation complète des sous-modules a été requise (`git submodules`).

L'environnement de développement choisi est **STM32CubeIDE**. Le SDK ne respectant pas la structure standard de cet IDE, le projet a été importé en tant que "Makefile Project with Existing Code", en ciblant spécifiquement le répertoire de la carte d'évaluation de l'émetteur (`mm-mm6108-ekh05`, avant sa mise à jour, `mm-mm8108-ekh05` après).

### 9.2 Adaptation de la chaîne de cross compilation

Sous Windows, les outils de compilation en ligne de commande tels que `make` ne sont pas natifs. Pour pallier ce problème, les variables d'environnement du projet dans STM32CubeIDE ont été modifiées pour inclure les chemins absolus vers les utilitaires internes de l'IDE (`make.exe` et `arm-none-eabi-gcc.exe`).

De plus, le SDK force la recherche de la chaîne de compilation GCC ARM dans les répertoires typiques de Linux (tels que `/opt/`). Il a donc été nécessaire de modifier le fichier de configuration principal `mkcore-arm-cortex-m33f.mk` pour empêcher cette recherche et permettre au système d'utiliser le `PATH` de Windows :

```
# --- MODIFICATION POUR WINDOWS / STM32CubeIDE ---
# On ignore la recherche des dossiers Linux /opt/...
# et on laisse le PATH systeme de STM32CubeIDE trouver le
#   compilateur.
TOOLCHAIN_DIR :=
TOOLCHAIN_BASE := arm-none-eabi-

CC := "$(TOOLCHAIN_BASE)gcc"
CXX := "$(TOOLCHAIN_BASE)g++"
AS := $(CC) -x assembler-with-cpp
OBJCOPY := "$(TOOLCHAIN_BASE)objcopy"
```

### 9.3 Configuration du débogage matériel (ST-LINK)

Pour permettre l'exécution pas à pas et la pose de breakpoint, le projet a été converti en "Projet ARM" dans l'IDE. La sonde de débogage utilisée est le **ST-LINK** intégré à la carte EKH05, communiquant via le protocole SWD (*Serial Wire Debug*).

FIGURE 4 – Configuration du serveur GDB ST-LINK dans STM32CubeIDE

Il est important de noter une limitation physique de cette architecture : la sonde USB (ST-LINK) ne peut maintenir qu'un seul tunnel de communication actif à la fois. L'utilisation simultanée de la console série (ex : PuTTY, Termit) et du débogueur GDB entraîne des conflits matériels.

### 9.4 Provisionnement et calibration radio (Config Store)

Une étape indispensable, distincte de la compilation du code C, est le **provisionnement** de la puce Wi-Fi. La puce Morse Micro ne possède pas de mémoire non volatile interne pour ses paramètres vitaux ; elle ne peut pas démarrer sans son micro-code (BCF) et ses paramètres régionaux (*Country Code*), stockés dans une zone mémoire spécifique du STM32 appelée le *Config Store*.

#### 9.4.1 Architecture de flashage

Le processus de flashage repose sur une architecture client-serveur :

1. **Le Serveur (OpenOCD)** : Il établit la connexion matérielle entre le port USB et le processeur ARM via la sonde ST-LINK.
2. **Le Client (Script Python)** : Il lit le fichier de configuration utilisateur (`config.hjson`), génère le binaire correspondant, et commande au serveur de l'écrire dans la mémoire Flash du STM32 (à l'adresse `0x08004000`).

#### 9.4.2 Résolution du conflit de signature et de version matérielle

Lors des premiers déploiements, une erreur bloquait la communication SPI entre le MCU et la puce Wi-Fi (`Transport init failed - Chip ID 0x0000`). Le code embarqué plan-tait dès l'initialisation du HAL (Hardware Abstraction Layer).

L'analyse de ce blocage a révélé un désaccord matériel : le code C et le fichier de configuration cachaient une dépendance stricte à l'ancienne plateforme (MM6108). Notre matériel étant la version MM8108, le binaire de calibration injecté était incorrect. Par une coïncidence extrêmement heureuse, Morse Micro a mis à jour son SDK public la semaine suivante, ajoutant officiellement le support de la cible `mm-mm8108-ekh05`.

Cette mise à jour a permis de rediriger le script vers le fichier de calibration exact de la puce (`bcbf_mf08651_us.mbin`). Combiné à l'ajustement du fichier `config.hjson` pour cibler la région "US", la puce radio a pu démarrer avec succès.



Le module a été sciemment configuré pour cibler la région "US" (États-Unis, norme FCC sur la bande des 915 MHz) au lieu du domaine européen par défaut (norme ETSI sur la bande des 868 MHz). Ce choix est basé par les contraintes de partage du spectre radio. La réglementation européenne impose des règles strictes telles que le *Listen Before Talk* (LBT) et des limites sévères de *Duty Cycle* (temps de transmission maximal). Ces mécanismes forcent le module radio à retenir ou espacer ses paquets pour écouter si le canal est libre.

Lors de nos tests initiaux, ces bridages réglementaires faisaient exploser la latence réseau de base à environ 670 ms. Le passage sur le standard américain permet de désactiver ces restrictions d'émission. Bien que cette configuration soit réservée à un cadre expérimental, elle était indispensable pour ce travail de Bachelor afin de mesurer les performances brutes et réelles de la puce (ramenant la latence radio autour de 3 à 5 ms, testée par ping), indépendamment des limitations légales.

## 10 Implémentation du réseau Wi-Fi HaLow

La norme IEEE 802.11 intègre intrinsèquement des mécanismes de fiabilité, tels que les accusés de réception (ACK) et les retransmissions automatiques, destinés à garantir l'intégrité des données. Cependant, dans le cadre d'un flux vidéo temps réel pour le FPV, la perte d'une trame est visuellement préférable à l'attente d'une retransmission, qui induit une latence beaucoup trop élevée. L'architecture réseau a donc été configurée pour s'affranchir de ces contraintes.

### 10.1 Stratégie de diffusion (UDP Broadcast et No-ACK)

Pour contourner l'attente des acquittements matériels de la couche MAC, l'application embarquée sur le STM32 utilise le protocole de transport UDP sur une adresse IP de diffusion *Broadcast*.

Dans le code C exploitant la pile LwIP, l'adresse de destination est configurée sur l'adresse de diffusion locale du sous-réseau :

```
IP4_ADDR(ip_2_ip4(&dest_ip), 192, 168, 12, 255);
```

En ciblant l'adresse .255, la couche réseau LwIP force l'en-tête Wi-Fi à utiliser l'adresse MAC de destination FF:FF:FF:FF:FF:FF. Selon la norme 802.11, les trames diffusées à cette adresse ne déclenchent aucun acquittement (ACK) de la part des récepteurs. L'émetteur expédie ainsi les paquets en flux tendu, ce qui annule la latence induite par les retransmissions au prix de potentielles pertes de paquets.

De plus, la taille de la charge utile (*Payload*) UDP a été fixée à 1400 octets :

```
struct pbuf *p = pbuf_alloc(PBUF_TRANSPORT, 1400, PBUF_REF);
```

Cette valeur approche la taille maximale d'une trame Ethernet standard (MTU), ce qui minimise le ratio d'en-têtes (Overhead MAC+IP+UDP) par rapport aux données utiles.

### 10.2 Optimisation de la réactivité radio : Désactivation du PSM

Par défaut, les modules Wi-Fi HaLow Morse Micro intègrent des mécanismes de gestion de l'énergie (*Power Saving Mode*) destinés aux capteurs IoT sur batterie. Ce mode place la radio en veille entre les transmissions, ce qui induisait dans nos tests une latence systématique d'environ **22 ms** par paquet.

Pour un usage FPV, cette économie d'énergie n'est pas acceptable. Le mode veille a été explicitement désactivé dans le firmware du STM32 via l'appel suivant :

```
mmwlan_set_power_save_mode(MMWLAN_PS_DISABLED);
```

Cette modification a permis de stabiliser le temps de réponse réseau (*ping*) entre **3 et 5 ms**, garantissant une gigue minimale.

### 10.3 Contournement des contraintes réglementaires (ETSI vs FCC)

Lors des premiers essais d'établissement de la liaison radio en MCS 1, une latence de base colossale d'environ 670 ms a été observée sur de simples paquets de test (Ping).

L'analyse de ce délai a mis en évidence l'impact restrictif des normes réglementaires européennes (ETSI) régissant la bande des 868 MHz. Ces normes imposent un mécanisme de partage des ondes strict appelé *Listen Before Talk* (LBT) ainsi que des restrictions de *Duty Cycle* (temps d'émission maximal autorisé). Le module radio retenait intentionnellement les paquets pour respecter ces seuils légaux.

Afin d'évaluer les performances brutes du changement de MCS sans ces bridages logiciels imposés par la région, la configuration de l'émetteur et du récepteur a été basculée sur le domaine réglementaire américain (FCC, bande des 915 MHz). L'utilisation du Canal 27 (915.5 MHz) a permis de désactiver les mécanismes LBT, ramenant immédiatement la latence réseau de base à environ 27 ms pour des paquets de 1450 octets.

### 10.4 Le routage et l'isolation IP (OpenWrt)

Pour éviter les conflits de routage sur la station de réception, le réseau FPV est strictement isolé sur le sous-réseau 192.168.12.x.

Lors de l'implémentation, un conflit IP majeur bloquait la transmission : le routeur OpenWrt (récepteur) possédait l'adresse 192.168.12.1 simultanément sur son interface Wi-Fi HaLow (ahwlan) et sur son interface Ethernet locale (lan). Pour résoudre ce conflit, il a suffi de migrer le LAN sur 192.168.13.1.

Concernant le transfert des données à travers le routeur, une première approche a été testée : les paquets UDP entrants sur l'interface sans-fil étaient interceptés (via `tcpdump`) puis redirigés manuellement vers le port Ethernet à l'aide de l'utilitaire `netcat` (`nc`). Cependant, ce traitement en espace utilisateur (*User-Space*) sollicitait lourdement le processeur du routeur et induisait une latence importante.

```
tcpdump -i wlan0 -U -s 0 -w - "udp port 1337" | nc 192.168.12.10 9999
```

Pour optimiser le routage, un pont logiciel (*Bridge*) a finalement été forcé au niveau du système d'exploitation du routeur :

```
brctl addif br-ahwlan wlan0
```

Ce pont matériel permet au flux *Broadcast* en provenance de l'interface sans-fil de traverser le routeur et d'être répliqué directement sur le câble Ethernet (pour autant que la station de réception connectée soit sur le bon sous réseau) sans être intercepté ni traité par la pile TCP/IP ou des utilitaires logiciels, économisant ainsi 10 à 20 ms de délai de traitement additionnel.

## 10.5 Validation empirique et mathématique de la bande passante vidéo

La désactivation de l'adaptation automatique de débit (*Rate Control*) a été effectuée dans le SDK via la fonction ATE (*Automated Test Equipment*, fonction utilisée en interne pour forcer des configurations matérielles spécifiques lors des phases de tests) :

```
mmwlan_ate_override_rate_control(MMWLAN_MCS_2, MMWLAN_BW_2MHZ,
MMWLAN_GI_NONE);
```

En forçant l'utilisation du **MCS 2** sur une largeur de bande de **2 MHz**, la modulation employée (QPSK) garantit une excellente robustesse face aux obstacles physiques. Des mesures empiriques (par capture `tcpdump`) ont validé un débit utile réseau stable d'environ **1.34 Mbps** (soit une moyenne de 120 paquets de 1400 octets par seconde).

Il est ensuite nécessaire de confronter cette capacité physique aux exigences du flux vidéo. Le débit brut (non compressé) se calcule ainsi :

$$D_{brut} = W \times H \times F \times B_{pp}$$

Afin de contourner la rétention d'image du processeur de signal (ISP) de la caméra, la fréquence d'échantillonnage a dû être fixée à 60 FPS. Pour le flux vidéo retenu en résolution 480p (soit  $640 \times 480$  pixels), capturé à 60 images par seconde et codé sur 24 bits de couleurs :

$$D_{brut\_480p60} = 640 \times 480 \times 60 \times 24 = 442\,368\,000 \text{ bps}$$

Soit environ 497.6 Mbps.

Un encodeur matériel moderne (H.264 / H.265), configuré pour du streaming en temps réel, offre un ratio de compression effectif standard avoisinant 1 :200. Le débit réseau théorique requis deviendrait alors :

$$D_{cible} = \frac{D_{brut}}{C_{ratio}}$$

$$D_{cible\_480p60} = \frac{442.4 \text{ Mbps}}{200} \approx 2.21 \text{ Mbps}$$

Cette modélisation démontre clairement qu'à 60 images par seconde, même une résolution modeste de 480p avec une compression standard exigerait environ 2.21 Mbps, ce qui saturerait immédiatement le canal radio imposé par le MCS 2 (limité empiriquement à 1.34 Mbps).

C'est pour cette raison critique que l'encodeur matériel doit être paramétré de manière drastique. Afin de forcer ce flux à traverser la liaison radio tout en conservant une marge de sécurité vitale face aux baisses imprévisibles de la qualité du lien RF en condition de vol NLOS, le taux de compression final de l'encodeur a été bridé de manière stricte (CBR - *Constant Bit Rate*) à **400 kbps**.

### 10.5.1 Impact de la compression sur la fragmentation des paquets

Ce compromis de 400 kbps impose à l'encodeur un ratio de compression extrême (environ 1 :1240) au détriment de la fidélité visuelle, mais garantit un avantage décisif sur la latence matérielle. En lissant ce débit sur la fréquence d'échantillonnage, le poids moyen d'une image vidéo compressée se calcule ainsi :

$$Taille_{image} = \frac{400\,000 \text{ bits/s}}{60 \text{ FPS} \times 8 \text{ bits}} \approx 833 \text{ octets}$$

Sachant que la charge utile maximale (MTU) définie pour nos trames UDP et nos transferts SPI est de **1400 octets**, l'écrasante majorité des images générées par l'encodeur tiennent dans **un seul et unique paquet réseau**. Cette caractéristique est fondamentale pour la basse latence : elle évite la fragmentation IP et exempte le récepteur de devoir stocker et attendre de multiples paquets pour reconstruire et décoder une seule trame vidéo.

## 11 Capture et traitement du flux vidéo

Une fois le pont réseau Sub-1 GHz établi, le défi consiste à formater la source vidéo pour qu'elle puisse traverser ce canal étroit sans générer de goulots d'étranglement. Cette section détaille les choix logiciels, l'optimisation des paramètres de compression, et la gestion des tampons matériels.

### 11.1 Arbitrage entre les normes H.264 et H.265

Le choix initial du projet s'était porté sur la norme **H.265 (HEVC)**. Successeur du H.264, elle offre une efficacité de compression supérieure de 50%, permettant de maintenir une image nette même à des débits très faibles (400 kbps). Cependant, cette complexité algorithmique exige des ressources de décodage importantes.

Lors des phases de tests sur la station de réception finale (Raspberry Pi 4), l'absence d'un pilote de décodage matériel H.265 stable et performant sous Linux (`v4l2h265dec`) a provoqué une latence très élevée, dépassant les 300 ms.

Le système a donc été *downgradé* vers la norme **H.264 (AVC)**. Bien que moins performante en termes de ratio qualité/débit, elle bénéficie d'un support matériel universel. Ce choix a permis d'exploiter un décodage extrêmement rapide sur Raspberry Pi, ramenant la latence de traitement au niveau de la milliseconde tout en conservant une fluidité d'image compatible avec le pilotage FPV.

### 11.2 Le framework GStreamer et l'optimisation H.264

Pour gérer l'acquisition, la compression et l'encapsulation de la vidéo, le framework *open-source* GStreamer a été retenu de bout en bout. Son architecture modulaire permet de relier directement la sortie d'un capteur matériel à un encodeur, en minimisant les copies en mémoire vive (*Zero-Copy*).

La compression du flux exploite donc la norme H.264. La configuration a été pensée pour prioriser la vitesse au détriment de l'efficacité de compression absolue. Trois paramètres critiques ont été forcés :

- **Le contrôle de débit strict (CBR - control-rate=1)** : Le débit est verrouillé à 400 kbps pour éviter tout pic de données soudain qui saturerait le bus SPI.
- **La désactivation des images bidirectionnelles (num-B-Frames=0)** : Très important pour la basse latence. L'encodeur n'a plus besoin d'attendre l'image suivante pour calculer l'image courante.
- **L'espacement des images clés (idrinterval=1000)** : L'envoi d'images clés (I-Frames), très volumineuses, est espacé au maximum. Limiter leur fréquence évite les rafales massives de paquets (*bursts*) qui saturent la radio.

#### 11.2.1 Précisions sur les paramètres de pipeline

Certains drapeaux (*flags*) spécifiques ont été ajoutés pour forcer le comportement temps réel de la chaîne :

- **disable-passsthrough=true** : Par défaut, un parseur peut laisser passer les données sans les traiter si le format semble correct. Ce flag force l'élément `h264parse`

à analyser activement chaque octet du flux pour reconstruire proprement les unités de décodage (NAL units). Cela garantit que le décodeur reçoit des trames parfaitement alignées, évitant toute attente inutile.

- `config-interval=-1` : Force l'envoi des informations de configuration (SPS/PPS) à chaque image clé, permettant au récepteur de reprendre le flux instantanément même en cas de perte de connexion temporaire.

## 11.3 Pipelines d'émission (TX)

Afin de valider cette configuration sur différentes architectures matérielles, les pipelines ont été adaptés :

```
# Sur Nvidia Jetson Nano (NVENC)
gst-launch-1.0 nvarguscamerasrc ! \
'video/x-raw(memory:NVMM),width=640,height=480,framerate=60/1' ! \
nvv4l2h264enc bitrate=400000 control-rate=1 preset-level=1 \
insert-sps-pps=true idrinterval=1000 maxperf-enable=1 num-B-Frames
=0 ! \
h264parse ! fdsink

# Sur Raspberry Pi 4 (V4L2)
gst-launch-1.0 v4l2src device=/dev/video0 ! \
'video/x-raw,width=640,height=480,framerate=60/1' ! \
v4l2h264enc extra-controls="controls,video_bitrate=400000,
video_b_frames=0" ! \
h264parse ! fdsink
```

## 11.4 Pipelines de réception et d'affichage (RX)

À la réception, le défi s'inverse : il faut décoder et afficher le plus vite possible. L'élément critique est `sync=false`, qui ordonne à GStreamer d'afficher chaque trame dès qu'elle est prête, en ignorant les horodatages.

```
# Sur Station PC Windows (Acceleration D3D11)
gst-launch-1.0 udpsrc port=1337 buffer-size=0 ! \
"video/x-h264,width=640,height=480,stream-format=byte-stream" ! \
h264parse disable-passthrough=true ! d3d11h264dec ! \
queue max-size-buffers=1 leaky=downstream ! \
d3d11videosink sync=false async=false max-latency=0

# Sur Station cible Raspberry Pi (Bare-Metal via KMS)
gst-launch-1.0 -v udpsrc address=0.0.0.0 port=1337 buffer-size=0 ! \
"video/x-h264,width=640,height=480,framerate=60/1,stream-format=
byte-stream" ! \
h264parse disable-passthrough=true config-interval=-1 ! v4l2h264dec
! \
queue max-size-buffers=1 leaky=downstream ! \
```

```
kmssink sync=false max-latency=0
```

## 11.5 Identification et contournement de la latence de l'ISP

Malgré ces optimisations, une latence visuelle globale persistait lors des premiers tests. L'analyse a prouvé qu'au moins une partie de la latence "fantôme" provenait de l'Image Signal Processor (ISP) matériel de la caméra.

Pour effectuer les calculs d'exposition automatique (AE) et de balance des blancs (AWB), l'ISP conserve systématiquement un certain nombre d'images brutes dans une file d'attente avant de les transmettre à l'encodeur, nous émettons l'hypothèse qu'il en conserve 4. Mathématiquement, pour un capteur configuré pour échantillonner à 24 images par seconde, la durée d'une image est de 41.6 ms. Le délai induit par les 4 buffers logiciels est donc de :

$$Latence_{ISP} = 4 \times \left( \frac{1000}{24} \right) \approx 166.4 \text{ ms}$$

Pour contourner ce verrouillage matériel massif, la solution implémentée consiste à dissocier la fréquence d'échantillonnage du capteur de la fréquence réseau. En forçant le capteur à capturer à 60 FPS, les 4 buffers de l'ISP sont purgés nettement plus vite ( $4 \times 16.6 \text{ ms} = 66.4 \text{ ms}$ ), réduisant instantanément la latence visuelle de 100 ms.

## 11.6 Outils de diagnostic réseau

En complément de GStreamer, des utilitaires standards ont été employés pour le diagnostic du flux. **Netcat** (**nc**) a été utilisé pour rediriger manuellement les paquets UDP lors du prototypage, tandis que **tcpdump** a permis de capturer les trames à la volée sur les interfaces sans-fil pour valider la non-fragmentation des paquets HaLow.



## 12 Interfaçage matériel et transfert SPI

Le transfert du flux vidéo compressé entre le processeur d'acquisition (Jetson / RPi) et le modem radio (STM32) s'effectue via le bus matériel SPI. Cette étape est très importante car elle envoie les données au MCU pour l'émission RF.

### 12.1 Identification du bus SPI matériel

Une attention particulière a dû être portée à la configuration physique de la liaison. Le SDK Morse Micro ne supportant nativement aucun périphérique standard en dehors de la puce Wi-Fi, le routage exact du bus SPI du microcontrôleur sur le connecteur d'extension n'était pas documenté dans le guide utilisateur de base.

Pour résoudre cette inconnue, une analyse du dossier de conception officiel de la carte **EKH05** [7] a été menée afin d'identifier théoriquement les broches physiques et les pistes reliées aux périphériques internes du processeur. Afin de valider de manière absolue ce schéma avant d'y connecter l'ordinateur d'acquisition, une méthodologie de vérification simple a été effectuée : une tension de 3,3 V a été appliquée sur les lignes d'entrée potentielles repérées, permettant de confirmer au multimètre la correspondance exacte du routage (1).

| Broche EKH05 | Port STM32 | Signal SPI | Description fonctionnelle                       |
|--------------|------------|------------|-------------------------------------------------|
| D10          | PE12       | NSS (nCS)  | <i>Not Chip Select</i> (Sélection de l'esclave) |
| D11          | PE15       | MOSI       | <i>Master Out Slave In</i> (Données entrantes)  |
| D12          | PE14       | MISO       | <i>Master In Slave Out</i> (Données sortantes)  |
| D13          | PE13       | SCK        | <i>Serial Clock</i> (Horloge)                   |

TABLE 1 – Routage du bus SPI sur la carte d'évaluation EKH05

### 12.2 Portage de l'application d'émission (De Python à C)

Initialement développé en Python pour valider le concept, le programme lisant la sortie de GStreamer pour l'injecter sur le bus SPI a été entièrement réécrit en C.

Ce portage répondait à trois impératifs de temps réel :

1. **Élimination de la gestion mémoire asynchrone** : Le *Garbage Collector* de Python induit des micro-pauses aléatoires incompatibles avec la stabilité d'un flux FPV.
2. **Désactivation des caches de l'OS** : En C, la fonction `setvbuf(pipe, NULL, _IONBF, 0)` a été utilisée pour forcer le descripteur de fichier reliant GStreamer au programme à fonctionner en mode "zéro-buffer" (flux tendu).
3. **Précision des horloges** : Remplacement des temporisations logicielles incertaines par des lectures de registres GPIO matériels quasi-instantanées.

### 12.3 Contrôle de flux matériel (Handshake SPI)

Le transfert s'effectue sur le bus SPI à une fréquence d'horloge de 5 MHz. Mathématiquement, l'envoi d'un paquet UDP contenant une charge utile de 1400 octets (soit 11 200 bits) à cette fréquence requiert un temps de transmission physique incompressible. Ce temps théorique se calcule ainsi :

$$T_{SPI} = \frac{1400 \times 8 \text{ bits}}{5\,000\,000 \text{ bits/s}} = 0.00224 \text{ s} = \mathbf{2.24 \text{ ms}}$$

L'architecture UDP étant configurée sans acquittement, si l'ordinateur enchaîne l'envoi de ces paquets de 2.24 ms trop rapidement, la mémoire RAM du microcontrôleur STM32 déborde (*Buffer Overrun*). Des morceaux d'images sont écrasés avant même d'atteindre le module Wi-Fi, provoquant de sévères artefacts visuels (effets de "bavure").

Pour pallier ce problème, un mécanisme de *Handshake* matériel a été implémenté via une broche de signalisation dédiée (ex : GPIO 78 sur Jetson vers PD15 sur STM32) :

```
// Extrait du controle de flux materiel (Cote Linux)
do {
    lseek(gpio_fd, 0, SEEK_SET);
    read(gpio_fd, &gpio_val, 1);
} while (gpio_val == '0'); // Bloque tant que le STM32 est occupe
```

Le processus est le suivant : le STM32 abaisse ce signal (niveau BAS) dès qu'il reçoit un paquet de 1400 octets via l'interruption SPI. Il ne le relève (niveau HAUT) que lorsque la couche réseau LwIP a fini de transférer le paquet au module Wi-Fi HaLow. Cette synchronisation garantit un flux vidéo propre, dictant la cadence de l'encodeur sans aucune perte interne au système.

## 13 Itérations matérielles et analyse des performances

Afin d'isoler avec précision les goulots d'étranglement du système et de valider l'architecture proposée, une série de mesures de performances a été menée. L'objectif de cette démarche itérative est de quantifier séparément le temps de transit matériel, le temps de transport réseau, puis le temps de traitement de l'image, afin d'établir un bilan "Glass-to-Glass" complet.

### 13.1 Profilage théorique de la chaîne d'émission

La première étape a consisté à profiler les temps de traitement incompressibles des composants d'émission.

#### 13.1.1 Temps d'encodage matériel (Jetson Nano)

Pour mesurer le temps de traitement de l'encodeur matériel Nvidia (`nvv4l2h264enc`), les outils de traçage natifs de GStreamer ont été employés via la variable d'environnement `GST_TRACERS="latency(flags=element)"`. L'analyse des journaux d'exécution a démontré que le composant matériel traite et compresse une trame vidéo en un temps extrêmement stable d'environ **1.2 ms**.

#### 13.1.2 Mesure au cycle d'horloge du traitement MCU (STM32)

Pour profiler le transfert SPI et l'encapsulation LwIP, les fonctions classiques (`HAL_GetTick()`) manquaient de résolution. Le code a donc exploité le composant matériel ARM CoreSight DWT (*Data Watchpoint and Trace*). Le registre `DWT->CYCCNT` s'incrémente à chaque coup d'horloge (160 MHz), offrant une résolution de 6.25 nanosecondes.

```
// Demarrage du chronometre DWT dans l'interruption SPI
dwt_start = DWT->CYCCNT;

// ... [Encapsulation pbuf_alloc et envoi udp_sendto] ...

// Arrêt du chronometre et calcul du temps en microsecondes
dwt_end = DWT->CYCCNT;
mcu_cycles = dwt_end - dwt_start;
uint32_t freq_mhz = SystemCoreClock / 1000000; // 160 MHz
uint32_t time_us = mcu_cycles / freq_mhz;
```

L'extraction de cette métrique a révélé un temps de traitement MCU ultra-court de **0.8 ms par paquet**.

#### 13.1.3 Bilan de la latence d'émission par image

À la suite de l'optimisation du débit à **400 kbps** pour une fréquence de **60 FPS**, le poids moyen d'une image compressée se situe théoriquement autour de **833 octets**.

Il est toutefois important de préciser que cette valeur n'est qu'une moyenne lissée. En réalité, la compression H.264 génère des trames de tailles très variables en fonction de

la dynamique de l'image. Lors d'une scène très statique, l'encodeur est extrêmement efficace et plusieurs images prédictives (P-Frames) peuvent théoriquement tenir dans un seul paquet de 1400 octets. À l'inverse, lors de mouvements brusques du drone ou lors de l'émission d'images clés (I-Frames), la quantité de données explose, ce qui oblige le système à fragmenter la trame sur plusieurs paquets SPI et UDP.

Cependant, dans un cas moyen, cette taille de 833 octets étant inférieure à la capacité maximale de nos paquets (MTU de 1400 octets), on peut considérer qu'une trame vidéo standard est transmise dans **un seul paquet** SPI et UDP. Le temps total d'émission moyen d'une image est donc simplifié :

$$Latence_{TX} = T_{Encodeur} + (1 \times (T_{SPI} + T_{MCU}))$$

$$Latence_{TX} = 1.2 \text{ ms} + (2.24 \text{ ms} + 0.8 \text{ ms}) = \mathbf{4.24 \text{ ms}}$$

Ce résultat de **4.24 ms** représente la limite physique d'émission de notre architecture matérielle actuelle pour la formation d'un paquet nominal. Il confirme que la transmission radio elle-même est extrêmement rapide, laissant la majeure partie du budget de latence disponible pour les processus d'acquisition.

## 13.2 Caractérisation physique du transport radio

Pour confronter cette théorie à la réalité et isoler la latence de transmission pure, une architecture de test à "déclencheur commun" a été mise en place, s'affranchissant des problèmes de synchronisation d'horloges réseau.

Deux Raspberry Pi 4B ont été utilisées (TX et RX), avec leurs masses reliées et un bouton poussoir connecté simultanément sur leur broche BCM 17. Lorsqu'on presse le bouton, les deux programmes C (exploitant `mmap` sur `/dev/gpiomem`) détectent le front montant au même instant. Le TX injecte un paquet de test via SPI, et le RX fige son chronomètre haute résolution à la réception du paquet UDP.

Les résultats d'un test en rafale (*Burst*) de 10 paquets consécutifs ont donné un temps total de 124 ms, soit une moyenne en flux de **12.47 ms** par paquet. Ce résultat empirique, très proche de la théorie, prouve que le transport radio Sub-1 GHz est extrêmement rapide.

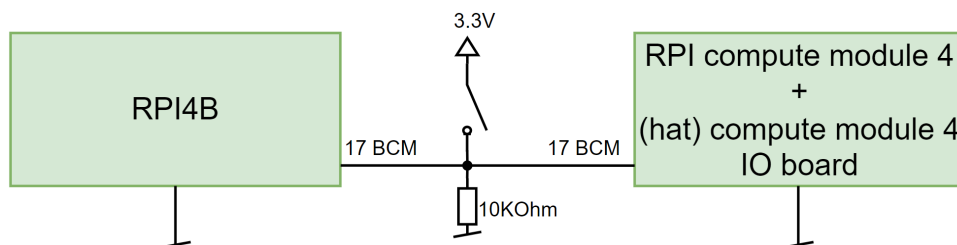


FIGURE 5 – Schéma du système de test

## 13.3 Isolement de la capture et abandon de la piste OpenHD

L'hypothèse suivante ciblait la pile logicielle de capture vidéo de l'OS. Le déploiement d'un système optimisé *ad hoc* comme OpenHD a été envisagé.

Avant d'engager cette lourde modification, un test de "boucle locale" (*loopback*) a été réalisé sur la Raspberry Pi 4B : capturer le flux caméra, l'encoder et l'afficher directement sur la même carte sans passer par le réseau. La latence mesurée fut de **5 à 7 ms**. Ce résultat a prouvé que la capture et la pile logicielle standard n'étaient pas le goulot d'étranglement. L'itération vers un OS personnalisé a donc été abandonnée.

### 13.4 Bilan "Glass-to-Glass" et latence résiduelle

La latence "Glass-to-Glass" absolue (entre un événement réel et son affichage sur l'écran final) a été mesurée entre **130 ms et 140 ms**.

L'élimination mathématique de la capture locale ( $\sim 7$  ms) et du transport radio ( $\sim 12.5$  ms) permet d'isoler un délai inexplicé de plus de 100 ms. Cette latence résiduelle provient de deux limitations inhérentes au matériel grand public :

1. **L'Image Signal Processor (ISP)** : Bien qu'il fût responsable d'une latence initiale importante (partiellement contournée par le suréchantillonnage du capteur à 60 FPS), le test de boucle locale a mesuré un délai de 5 à 7 ms. Ce résultat démontre que la puce matérielle de l'ISP, prise isolément, est capable de traiter le flux très rapidement. Toutefois, ce test s'effectuant sur une architecture locale, il est possible qu'il court-circuite certaines étapes d'interfaçage mémoire et de sérialisation propres à notre chaîne de transmission réseau. Bien que l'ISP ne semble pas être le goulot d'étranglement, la transition de ses données vers les tampons d'émission (pipeline réseau) pourrait cacher une latence d'intégration qui reste à investiguer.
2. **Le pipeline de réception logiciel** : Le parseur GStreamer (`h264parse`) doit retenir une trame entière pour valider l'intégrité du flux UDP brut. De plus, les gestionnaires de fenêtres des stations de diagnostic (Windows DWM ou X11) imposent souvent un *Triple Buffering* (synchronisation verticale), ajoutant mécaniquement 50 à 100 ms de latence incompressible avant l'affichage sur les écrans standards.

Le système matériel embarqué et le pont Wi-Fi HaLow ont donc été optimisés à leur limite absolue. La latence restante n'est pas le fait du réseau Sub-1 GHz, mais bien des couches d'abstraction d'acquisition et d'affichage.

### 13.5 Bilan récapitulatif des temps de traitement

Les différentes itérations matérielles et les mesures de profilage ont permis d'établir une cartographie précise des temps de traitement de chaque maillon de la chaîne, indépendamment des buffers d'affichage.

- **Encodage matériel (Jetson / RPi)** : Les outils de traçage estiment l'encodage H.264 d'une trame à environ **1.2 ms** sur Jetson, et **2 ms** sur Raspberry Pi.
- **Transfert SPI** : Le transfert d'un paquet de 1400 octets à 5 MHz requiert environ **2.24 ms**.
- **Traitement MCU (STM32)** : Le temps d'encapsulation LwIP, mesuré au cycle d'horloge, est de **0.8 à 1 ms**.

- **Temps de vol radio (Air time)** : Après la désactivation des modes d'économie d'énergie (*Power Saving Mode*) sur le module EKH19, qui induisaient artificiellement une latence de 22 ms, le temps de réponse réseau pur (Ping) entre le PC de réception et le relais s'est stabilisé entre **3 et 5 ms**.
- **Décodage matériel (RX)** : Les temps de décodage sont très similaires d'une plateforme à l'autre : entre **0.5 et 1 ms** sur le GPU du PC portable, et environ **2 à 3 ms** sur la Raspberry Pi 4.

Ce bilan théorique (Encodage + SPI + MCU + Air time + Décodage) donne une somme d'environ **8 à 10 ms** :

$$1.2 \text{ (Enc.)} + 2.24 \text{ (SPI)} + 0.8 \text{ (MCU)} + 3.0 \text{ (Air)} + 1.0 \text{ (Dec.)} = \mathbf{8.24 \text{ ms}}$$

Ce résultat mathématique est cohérent avec la mesure physique de **12.47 ms** obtenue lors du test en rafale avec le déclencheur matériel (bouton partagé entre les deux RPi). La différence d'environ 4 millisecondes s'explique par les différents *overhead* logiciel, notamment du noyau Linux (gestion de la file d'attente réseau et ordonnancement de l'OS) qui n'est pas pris en compte dans le calcul matériel pur. De même, le test de boucle locale (capture et affichage sur la même carte) mesuré entre 5 et 7 ms valide notre estimation d'encodage et de décodage couplés à ce même *overhead* de l'OS.

### 13.6 Bilan "Glass-to-Glass" et perspectives (Rendu intermédiaire)

Malgré la rapidité démontrée du pont réseau Sub-1 GHz, la latence "Glass-to-Glass" absolue (le délai entre un événement réel et son affichage sur l'écran du pilote) mesurée lors de nos derniers essais se situe entre **130 ms et 140 ms** (sur une architecture combinant RPi 4B et RPi Compute Module).

Ce décalage met en évidence une latence inexplicée d'environ 120 ms. Dans le cadre de ce rendu intermédiaire, la source exacte de cette latence résiduelle reste à identifier avec certitude.

Plusieurs hypothèses guideront la suite de ce travail de Bachelor :

1. **Goulot d'étranglement sous charge (MCU)** : Une hypothèse supposait qu'un bombardement massif de paquets vidéo ralentissait le STM32 par rapport à un simple test de ping. Cependant, si le traitement MCU était le goulot d'étranglement, les mémoires tampons déborderaient et la latence s'accumulerait indéfiniment au fil du temps. L'image restant stable, cette piste semble peu probable.
2. **Les buffers réseaux et logiciels (RX/TX)** : L'ISP semblant hors de cause suite au test de boucle locale, l'empilement logiciel lors de la mise en réseau (les files d'attente UDP du noyau Linux, le *jitter buffer* de GStreamer, et l'élément *h264parse* qui exige une trame complète) devient le suspect principal de cette rétention de 120 ms (des tests supplémentaires concernant d'éventuelles latences liées à l'ISP seront néanmoins nécessaires).
3. **L'encapsulation logicielle et l'affichage (RX)** : Bien que le test du bouton ait validé la pile réseau bas niveau du récepteur, l'empilement logiciel GStreamer (*h264parse*) et la synchronisation verticale des écrans de restitution restent des sources potentielles de rétention d'image.

La suite de ce projet se concentrera sur l'isolation stricte de ces dernières variables, afin de cerner l'origine de ces 120 millisecondes et de déterminer les optimisations ultimes possibles pour rapprocher ce système des standards de l'industrie FPV.

## 14 Conclusion intermédiaire

Le pilotage de drones FPV (*First Person View*) en environnement complexe se heurte depuis toujours à l'incapacité des fréquences standards (2.4 GHz et 5.8 GHz) à pénétrer efficacement les obstacles physiques. Ce travail de Bachelor vise à évaluer une solution de rupture technologique en détournant la norme IEEE 802.11ah (Wi-Fi HaLow), opérant dans la bande Sub-1 GHz, pour y faire transiter un flux vidéo en temps réel.

À ce stade d'avancement du projet, l'architecture fondamentale du système est non seulement établie, mais ses performances ont été démontrées et mesurées de manière rigoureuse.

### 14.1 Bilan des réalisations

La première grande réussite de ce travail réside dans la maîtrise de la liaison radio HaLow. En contournant les mécanismes d'acquittement de la couche MAC (utilisation de l'UDP Broadcast / mode No-ACK) et en modifiant les configurations de routage, nous avons garanti un canal de transmission fluide et déterministe.

La conception d'un pont de communication matériel (SPI vers Wi-Fi) sur un microcontrôleur STM32, couplé à un mécanisme de *handshake*, a permis de synchroniser le trafic vidéo sans perte. L'approche itérative adoptée a été déterminante : l'utilisation initiale d'une plateforme surpuissante (Nvidia Jetson Nano) a permis de certifier les temps d'encodage via le traçage logiciel GStreamer et d'écarter tout doute sur les performances de compression.

Fort de cette validation de base, l'architecture a évolué pour se rapprocher du matériel. La configuration retenue pour nos tests de bout en bout les plus récents repose ainsi sur des plateformes ARM : une Raspberry Pi 4B en émission (TX) et une *Compute Module 4* en réception (RX). L'analyse des latences des différents éléments de cette chaîne, confirmée par un test de déclencheur physique (bouton partagé), prouve que le transport pur (transfert SPI, traitement MCU, temps de vol RF) s'effectue en **12 à 13 millisecondes**. La technologie Sub-1 GHz n'est donc en rien un frein à la réactivité requise pour le pilotage.



## 14.2 Objectifs pour la phase finale

Si le lien radio a démontré d'excellentes performances intrinsèques sur des bancs de test isolés, la latence "Glass-to-Glass" absolue mesurée actuellement sur le flux vidéo complet se situe entre **130 ms et 140 ms**. Ce décalage met en évidence une latence résiduelle importante d'environ 120 ms, dont l'origine exacte reste à identifier et à prouver formellement.

Les hypothèses de travail actuelles s'orientent principalement vers les extrémités de la chaîne : les files d'attente internes de l'Image Signal Processor (ISP) de la caméra à l'acquisition (malgré les tests en boucle locale), ainsi que les tampons logiciels de reconstruction réseau et la synchronisation verticale inhérente aux pilotes d'affichage à la restitution. Bien que la faible latence mesurée lors du test de rebouclage vidéo semble écarter cette dernière option, des investigations supplémentaires resteront nécessaires pour l'exclure définitivement. Toutefois, le comportement du pont de communication lui-même (MCU et puce HaLow) sous une charge vidéo prolongée et en conditions RF dégradées nécessitera également d'être poussé plus loin pour écarter définitivement la piste d'un goulot d'étranglement réseau dynamique.

Les semaines restantes avant le rendu final de ce travail de Bachelor seront donc dédiées à l'identification formelle de chaque source de latence résiduelle dans le système. L'enjeu consistera à concevoir de nouveaux protocoles de mesure pour isoler chaque sous-système, de la lentille de la caméra jusqu'à l'écran. L'objectif ultime sera d'appliquer les optimisations nécessaires (contournement de l'ISP, affichage *Bare-Metal* strict, tests de stress réseau) afin de gommer ce délai, rapprochant ainsi le système des standards FPV et validant la viabilité du Wi-Fi HaLow pour l'exploration par drone.

Nathan Tschantz



## 15 Note sur l'utilisation de l'intelligence artificielle

Conformément aux directives académiques, l'auteur déclare avoir utilisé des outils d'intelligence artificielle générative (notamment Gemini 3 Flash et Pro) durant la phase de rédaction de ce rapport intermédiaire.

L'utilisation de cet outil a été strictement encadrée et ciblée sur les points suivants :

- **Assistance à la mise en page et structuration LaTeX** : Génération de structures de code (tableaux, indexation, glossaire) et résolution d'erreurs de compilation.
- **Aide à la formulation et synthèse** : Aide à la structuration logique du document (mise en place de l'approche *Bottom-Up*), et reformulation de notes de laboratoire brutes pour améliorer la clarté et la fluidité académique de la rédaction.
- **Optimisation et génération de blocs de code** : Co-rédaction et ajustement de certaines portions de code spécifiques (telles que les configurations de pipelines GStreamer ou des structures en C), ainsi que l'assistance lors de la réalisation de scripts ou de code (C ou Python). Ces éléments ont été générés systématiquement sur la base des propositions algorithmiques et des contraintes logiques dictées par l'auteur.
- **Soutien à la recherche documentaire** : Aide à l'analyse et à la navigation dans le code source du pilote Morse Micro.

*Note importante : L'intégralité de l'ingénierie système, de l'architecture globale, de la démarche itérative, de la conception des bancs de test matériels, des protocoles de mesure et des méthodologies de profilage n'a fait l'objet d'aucune génération par IA et provient exclusivement des travaux de l'auteur. De même, l'analyse des journaux d'exécution (logs) du noyau Linux a été réalisée et validée personnellement. Les requêtes (prompts) soumises à l'IA étaient systématiquement fondées sur ces recherches de bas niveau, des données brutes réelles et sur les notes de travail journalières issues des discussions avec Monsieur Mahboob Karimian (Chargé de R&D, Institut IICT, HEIG-VD).*

## 16 Bibliographie

### Références

- [1] CaddxFPV. Walksnail moonlight kit - spécifications techniques (latence fpv). <https://www.caddxfpv.com/products/walksnail-moonlight-kit-only>, 2026. Accédé le 17 mai 2026.
- [2] DeepBlueMbedded. Stm32 delay microsecond & millisecond utility (dwt delay). <https://deepbluembedded.com/stm32-delay-microsecond-millisecond-utility-dwt-delay-timer-delay/>, 2026. Accédé le 17 mai 2026.
- [3] DJI. Dji o4 air unit pro - spécifications techniques (latence de transmission). <https://store.dji.com/ch/product/dji-o4-air-unit-pro>, 2026. Accédé le 17 mai 2026.
- [4] GStreamer Project. GStreamer : Open source multimedia framework. <https://gstreamer.freedesktop.org/>, 2026. Accédé le 17 mai 2026.
- [5] Morse Micro. *Morse Micro OpenWrt 2.8 Web UI User Guide*. Morse Micro, 2024. Documentation technique constructeur.
- [6] Morse Micro. *MM8108-EKH05 User Guide (MM8108-EKH05-User-Guide.pdf)*. Morse Micro, 2025. Guide utilisateur du module d'évaluation EKH05.
- [7] Morse Micro. *MM8108-EKH05 V2 Design Package (Schematics, Layout and Pinout)*. Morse Micro, 2025. Accédé le 17 mai 2026.
- [8] Morse Micro. *MM8108-EKH19 User Guide (MM6108\_MM8108-Eval-Kit-User-Guide-2.8.pdf)*. Morse Micro, 2025. Guide utilisateur du module d'évaluation EKH19.
- [9] Morse Micro. Morse micro iot sdk framework. <https://github.com/MorseMicro/mm-iot-sdk>, 2026. Accédé le 17 mai 2026.
- [10] Morse Micro. Morse micro openwrt feed and configuration. <https://github.com/MorseMicro/openwrt>, 2026. Accédé le 17 mai 2026.
- [11] Morse Micro. Morse micro wi-fi halow driver source code. [https://github.com/MorseMicro/morse\\_driver](https://github.com/MorseMicro/morse_driver), 2026. Accédé le 17 mai 2026.
- [12] NVIDIA Corporation. Getting started with jetson nano developer kit. <https://developer.nvidia.com/embedded/learn/get-started-jetson-nano-devkit>, 2026. Accédé le 17 mai 2026.
- [13] Pi4J Project. Raspberry pi compute module 4 pinout. <https://www.pi4j.com/1.3/pins/rpi-cm4.html>, 2026. Accédé le 17 mai 2026.
- [14] Raspberry Pi Ltd. Raspberry pi documentation - getting started. <https://www.raspberrypi.com/documentation/computers/getting-started.html>, 2026. Accédé le 17 mai 2026.
- [15] STMicroelectronics. Stm32u585vit6q datasheet & spécifications. <https://eu.mouser.com/ProductDetail/STMicroelectronics/STM32U585VIT6Q>, 2026. Accédé le 17 mai 2026.
- [16] Wikipédia. Ieee 802.11ah. [https://fr.wikipedia.org/wiki/IEEE\\_802.11ah](https://fr.wikipedia.org/wiki/IEEE_802.11ah), 2026. Accédé le 17 mai 2026.

## A Guide de connexion et d'accès aux équipements

Durant la phase de développement, le système étant distribué sur plusieurs plateformes matérielles dépourvues d'écran (architecture *headless*), des méthodes de connexion spécifiques ont été mises en place pour interagir avec chaque équipement.

### A.1 Accès au microcontrôleur d'émission (STM32 / EKH05)

Le module Morse Micro EKH05 intègre un débogueur ST-LINK natif. La connexion physique s'effectue via un simple câble micro-USB relié à la station de développement (PC Windows). Cette liaison fournit deux interfaces virtuelles simultanées :

- **Le flashage et débogage (SWD)** : Géré de manière transparente par STM32CubeIDE via le serveur OpenOCD.
- **La console de logs (UART)** : Accessible via un émulateur de terminal série (tel que PuTTY). Les paramètres de connexion au port COM virtuel sont systématiquement : Baudrate 115200, 8 bits de données, 1 bit d'arrêt, aucune parité.

Du point de vue du réseau HaLow, le STM32 possède l'adresse IP source 192.168.12.2.

### A.2 Accès aux ordinateurs de capture (Jetson Nano / RPi 4B)

Pour modifier les scripts de capture vidéo et lancer les pipelines GStreamer sans alourdir le processeur avec une interface graphique, les cartes d'acquisition ont été configurées pour un accès distant via SSH (*Secure Shell*).

**Via point d'accès mobile ou réseau local** : La connexion s'effectue depuis le PC de développement. Par exemple, pour la Jetson connectée sur un réseau partagé :  
`ssh jetsontb@10.120.243.168`

**Via câble Ethernet direct** : En cas de déploiement en extérieur (sans réseau local), la carte est configurée avec une adresse IP statique de repli sur son port Ethernet pour la prise en main.

### A.3 Accès au routeur relais de réception (GL.iNet MT3000 / OpenWrt)

Le routeur OpenWrt faisant office de pont réseau (Bridge) est le centre névralgique de la réception au sol. Son adresse IP sur l'interface sans-fil est 192.168.12.1. Son administration s'effectue exclusivement via son interface Ethernet (LAN), qui a été isolée pour éviter tout conflit de routage.

- **Interface Web (LuCI)** : Accessible depuis un navigateur web à l'adresse `http://192.168.13.1`. Elle permet de configurer le mode moniteur ou les paramètres de l'AP sans chiffrement.
- **Accès SSH (Ligne de commande)** : Indispensable pour injecter le module noyau (`morse.ko`) et créer le pont logiciel (`brctl addif br-ahwlan wlan0`). La connexion s'effectue via :  
`ssh root@192.168.13.1`

## A.4 Accès et adressage des stations de réception (RX)

Pour éviter les conflits de routage avec le système d'exploitation de la station de réception, le réseau FPV est strictement isolé sur le sous-réseau 192.168.12.x, sans passerelle par défaut (aucun accès Internet croisé). L'adresse de diffusion (*Broadcast*) ciblée par l'émetteur est 192.168.12.255.

**Station de diagnostic (PC Windows) :** L'interface réseau Ethernet doit être fixée manuellement sur l'adresse 192.168.12.10 (Masque : 255.255.255.0, Passerelle : Vide (ou 192.168.12.1)).

**Station cible (RPi Compute Module / RPi 4B) :** Lors de la réception "Bare-Metal" sous Linux, l'interface Ethernet nécessite également d'être forcée manuellement sur une adresse statique pour recevoir le flux ponté par le routeur :  
IP : 192.168.12.50

## A.5 Tableau récapitulatif d'adressage et d'accès

Le tableau 2 résume l'ensemble du plan d'adressage et des interfaces de gestion de l'architecture déployée :

| Équipement / Rôle                                     | Interface d'accès                  | Adresse IP FPV                                   | Remarques                                                                  |
|-------------------------------------------------------|------------------------------------|--------------------------------------------------|----------------------------------------------------------------------------|
| <b>STM32 / EKH05</b><br>(Modem Émetteur)              | USB (UART / SWD)                   | 192.168.12.2                                     | Target des trames SPI. Diffuse vers l'adresse .255.                        |
| <b>Jetson Nano / RPi 4B</b><br>(Acquisition Vidéo TX) | SSH (Wi-Fi dev ou Ethernet direct) | <i>Non applicable</i>                            | Connecté en SPI au modem. Pas d'IP sur le réseau FPV.                      |
| <b>GL.iNet MT3000</b><br>(Routeur Relais RX)          | SSH / Web (LuCI) via LAN           | 192.168.12.1<br>(Interface <code>ahwlan</code> ) | IP d'administration LAN décalée sur 192.168.13.1 pour éviter les conflits. |
| <b>PC Windows</b><br>(Station de diag. RX)            | Accès local                        | 192.168.12.10                                    | Passerelle laissée vide. (ou 192.168.12.1)                                 |
| <b>RPi Compute Module 4</b><br>(Station cible RX)     | SSH (Wi-Fi dev)                    | 192.168.12.50                                    | Fixée manuellement sur l'interface Ethernet ( <code>eth0</code> ).         |

TABLE 2 – Récapitulatif des accès et du plan d'adressage (Sous-réseau FPV : 192.168.12.x)

## B Modifications du pilote Morse Micro

Afin de forcer l'utilisation d'une modulation robuste (MCS 2) et l'activation du code correcteur d'erreurs (LDPC) en dehors de l'interface standard d'OpenWrt, des modifications directes (Patching) ont été opérées sur le code source C du module noyau (Kernel Driver) de la puce MM8108.

Les modifications ont porté sur les fichiers sources lors de la cross-compilation du noyau Linux pour l'architecture MediaTek Filogic :

- `command.c` : Surcharge de la structure `morse_cmd_req_set_transmission_rate` pour forcer les bits `req.use_ldpc = 1` et injecter le `mcs_index`.
- `usb.c` : Injection de l'appel manuel `morse_cmd_set_fixed_transmission_rate(mors, 1, 2, 0, 0)` directement dans la séquence d'initialisation `morse_usb_probe` au montage de l'interface.

L'ensemble de ces modifications a permis de compiler un module `morse.ko` personnalisé, assurant la stabilité de la liaison unidirectionnelle nécessaire au transport UDP Broadcast.

## C Mesure de latence via bouton partagé

### C.1 Le Code TX (Émetteur de test : `latency_tx.c`)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <linux/spi/spidev.h>
#include <sys/mman.h>

#define CHUNK_SIZE 1400
#define SPI_DEVICE "/dev/spidev0.0"

#define GPIO_BTN 17 // Pin physique 11 (Le Bouton)
#define GPIO_HANDSHAKE 24 // Pin physique 18 (Le STM32)

volatile unsigned *gpio;

void setup_gpiomem() {
    int mem_fd = open("/dev/gpiomem", O_RDWR | O_SYNC);
    if (mem_fd < 0) { perror("Erreur_gpiomem"); exit(1); }
    gpio = (volatile unsigned *)mmap(NULL, 4096, PROT_READ |
        PROT_WRITE, MAP_SHARED, mem_fd, 0);
    close(mem_fd);

    *(gpio + 2) &= ~(7 << 12); // GPIO 24 en ENTREE
```

```

    *(gpio + 1) &= ~(7 << 21); // GPIO 17 en ENTREE
}

int main() {
    int spi_fd;
    uint32_t speed = 5000000; // 5 MHz
    uint8_t bits = 8;
    uint32_t mode = 0;

    setup_gpiomem();

    spi_fd = open(SPI_DEVICE, O_RDWR);
    if (spi_fd < 0) { perror("SPI open"); return 1; }

    ioctl(spi_fd, SPI_IOC_WR_MODE, &mode);
    ioctl(spi_fd, SPI_IOC_WR_BITS_PER_WORD, &bits);
    ioctl(spi_fd, SPI_IOC_WR_MAX_SPEED_HZ, &speed);

    uint8_t buffer[CHUNK_SIZE];
    memset(buffer, 0xAA, CHUNK_SIZE);

    struct spi_ioc_transfer tr = {
        .tx_buf = (uint64_t)(uintptr_t)buffer,
        .rx_buf = 0,
        .len = CHUNK_SIZE,
        .speed_hz = speed,
        .bits_per_word = bits,
        .cs_change = 0,
    };

    int test_step = 0;

    while (1) {
        if (test_step < 10) {
            printf("\n>>> [UNITAIRE_%d/10] Appuie sur le bouton  
pour envoyer 1 paquet...\n", test_step + 1);
        } else {
            printf("\n>>> [BURST_TEST] Appuie sur le bouton pour  
envoyer la RAFALE de 10 paquets...\n");
        }

        // 1. Attente du BOUTON
        while ( (*(gpio + 13) & (1 << GPIO_BTN)) == 0 );

        if (test_step < 10) {
            // --- ENVOI UNITAIRE ---
            int timeout = 0, stm_ready = 1;
            while ( (*(gpio + 13) & (1 << GPIO_HANDSHAKE)) == 0 ) {

```

```

        if (timeout++ > 5000000) { stm_ready = 0; break; }
    }
    if (stm_ready) {
        ioctl(spi_fd, SPI_IOC_MESSAGE(1), &tr);
        printf("_->_Paquet_envoye.\n");
        test_step++;
    } else {
        printf("[ERREUR]_STM32_bloque.\n");
    }
} else {
    // --- ENVOI EN RAFALE (BURST) ---
    int paquets_ok = 0;
    for (int i = 0; i < 10; i++) {
        int timeout = 0, stm_ready = 1;
        while ( (*(gpio + 13) & (1 << GPIO_HANDSHAKE)) == 0 ) {
            if (timeout++ > 5000000) { stm_ready = 0; break; }
        }
        if (!stm_ready) {
            printf("[ERREUR]_STM32_a_plante_au_paquet_%d.\n", i+1);
            break;
        }
        ioctl(spi_fd, SPI_IOC_MESSAGE(1), &tr);
        paquets_ok++;
        usleep(100); // 100us pour laisser le STM32 reagir
    }
    if (paquets_ok == 10) {
        printf("_->_Rafale_de_10_paquets_envoyee!\n");
        test_step = 0; // Reinitialise le cycle de test
    }
}

// Anti-rebond du bouton
usleep(500000);
}

close(spi_fd);
return 0;
}

```

## C.2 Le Code RX (Chronomètre de précision : latency\_rx.c)

```

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

```



```
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <time.h>
#include <pthread.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define GPIO_BTN 17
#define PORT 1337

volatile unsigned *gpio;
volatile uint64_t t_start = 0;
volatile uint64_t t_end = 0;
volatile int packet_count = 0;

void setup_gpiomem() {
    int mem_fd = open("/dev/gpiomem", O_RDWR | O_SYNC);
    if (mem_fd < 0) { perror("Erreur_gpiomem"); exit(1); }
    gpio = (volatile unsigned *)mmap(NULL, 4096, PROT_READ |
        PROT_WRITE, MAP_SHARED, mem_fd, 0);
    close(mem_fd);
    *(gpio + 1) &= ~(7 << 21); // GPIO 17 en ENTREE
}

uint64_t get_time_ns() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
    return (uint64_t)ts.tv_sec * 1000000000ULL + ts.tv_nsec;
}

// Thread reseau
void* udp_listener(void* arg) {
    int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in servaddr = {0};
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(PORT);
    bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(
        servaddr));

    uint8_t buffer[2000];

    while(1) {
        recvfrom(sockfd, (char *)buffer, sizeof(buffer), 0, NULL,
            NULL);
        t_end = get_time_ns(); // Mise a jour de l'heure d'arrivee
    }
}
```

```

        packet_count++;           // Incrmente le compteur de paquets
        recurs
    }
    return NULL;
}

int main() {
    setup_gpiomem();

    pthread_t thread_id;
    pthread_create(&thread_id, NULL, udp_listener, NULL);

    int test_step = 0;
    double sum_latencies = 0;

    printf(">>>RX_CHRONO_PRET. Initialisation...\n");

    while (1) {
        // Preparation des variables partagees avant l'appui du
        // bouton
        packet_count = 0;

        // 1. Attente du BOUTON
        while ( (*(gpio + 13) & (1 << GPIO_BTN)) == 0 );

        // 2. Lancement du chrono
        t_start = get_time_ns();

        int target_packets = (test_step < 10) ? 1 : 10;

        // 3. Attente reseau avec Timeout
        while (packet_count < target_packets) {
            if (get_time_ns() - t_start > 1500000000ULL) { //
                Timeout 1.5s
                printf("[ERREUR] Timeout! Seulement %d/%d paquet(s)
                    )recu(s).\n", packet_count, target_packets);
                test_step = 0;
                sum_latencies = 0;
                break;
            }
        }

        // 4. Calculs et Affichage
        if (packet_count == target_packets) {
            double latency_ms = (t_end - t_start) / 1000000.0;

            if (test_step < 10) {
                // Phase 1 : Test unitaire
            }
        }
    }
}

```

```

        printf("Test%d/10|Latence: %.3fms\n",
               test_step + 1, latency_ms);
        sum_latencies += latency_ms;

        if (test_step == 9) {
            printf("
            -----\n
            ");
            printf("MOYENNE UNITAIRE: %.3fms\n",
                   sum_latencies / 10.0);
            printf("
            -----\n
            ");
        }
        test_step++;
    } else {
        // Phase 2 : Test en rafale
        printf("=====\\n");
        printf("LATENCE BURST TOTALE (10 paquets): %.3fms\\n", latency_ms);
        printf("MOYENNE EN FLUX (Latence / 10) : %.3fms\\n", latency_ms / 10.0);
        printf("=====\\n\\n");

        // Fin du cycle, on repart a zero
        test_step = 0;
        sum_latencies = 0;
    }
}

// Anti-rebond
usleep(500000);
}
return 0;
}

```

## D Code TX

### D.1 JETSON (jetson\_to\_rpi.c)

```

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>

```

```
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <linux/spi/spidev.h>

#define CHUNK_SIZE 1400
#define SPI_DEVICE "/dev/spidev0.0"
#define GPIO_VALUE_PATH "/sys/class/gpio/gpio78/value"
#define GPIO_EXPORT_PATH "/sys/class/gpio/export"
#define GPIO_DIR_PATH "/sys/class/gpio/gpio78/direction"

// Fonction pour initialiser le GPIO 78 proprement
void setup_gpio() {
    int fd;

    fd = open(GPIO_EXPORT_PATH, O_WRONLY);
    if (fd >= 0) {
        write(fd, "78", 2);
        close(fd);
    }

    usleep(100000);

    // direction = "in"
    fd = open(GPIO_DIR_PATH, O_WRONLY);
    if (fd >= 0) {
        write(fd, "in", 2);
        close(fd);
    }
}

int main() {
    int spi_fd, gpio_fd;
    uint32_t mode = 0;
    uint32_t speed = 5000000; // 5 MHz
    uint8_t bits = 8;

    printf(">>> Initialisation du GPIO 78...\n");
    setup_gpio();

    gpio_fd = open(GPIO_VALUE_PATH, O_RDONLY);
    if (gpio_fd < 0) {
        perror("Erreur: Impossible d'ouvrir le GPIO 78");
        return 1;
    }

    printf(">>> Initialisation du SPI...\n");
    spi_fd = open(SPI_DEVICE, O_RDWR);
```

```

if (spi_fd < 0) {
    perror("Erreur: Impossible d'ouvrir le SPI");
    return 1;
}

ioctl(spi_fd, SPI_IOC_WR_MODE, &mode);
ioctl(spi_fd, SPI_IOC_WR_BITS_PER_WORD, &bits);
ioctl(spi_fd, SPI_IOC_WR_MAX_SPEED_HZ, &speed);

// 720p width=1280,height=720
// 1080p width=1920,height=1080
// cette commande peut donc varier en fonction des essais
// video
const char *gst_cmd = "gst-launch-1.0 -q nvarguscamerasrc
    aelock=true awblock=true tnr-mode=0 ee-mode=0!
    \"video/x-raw(memory:NVMM),width=640,height
    =480,format=NV12,framerate=30/1\"!
    \"nvv4l2h264enc bitrate=400000 control-rate=1
    preset-level=1 maxperf-enable=1\"
    \"profile=0 insert-sps-pps=true idrinterval
    =1000 num-B-Frames=0\"
    \"slice-header-spacing=1400 bit-packetization
    =1!\"
    \"h264parse! fdsink fd=1 sync=false blocksize
    =1400";

printf(">>> Demarrage de GStreamer...\n");
FILE *pipe = popen(gst_cmd, "r");
if (!pipe) return 1;

setvbuf(pipe, NULL, _IONBF, 0);

uint8_t buffer[CHUNK_SIZE];
struct spi_ioc_transfer tr = {
    .tx_buf = (unsigned long)buffer,
    .rx_buf = 0,
    .len = CHUNK_SIZE,
    .speed_hz = speed,
    .bits_per_word = bits,
};

printf(">>> Flux video actif! Attente du signal STM32...\n");

char gpio_val;

while (1) {
    size_t bytes_read = fread(buffer, 1, CHUNK_SIZE, pipe);
    if (bytes_read == 0) break;
}

```

```

        if (bytes_read < CHUNK_SIZE) {
            memset(buffer + bytes_read, 0, CHUNK_SIZE - bytes_read)
                ;
        }
        int stuck_counter = 0;
        do {
            lseek(gpio_fd, 0, SEEK_SET);
            if (read(gpio_fd, &gpio_val, 1) != 1) {
                gpio_val = '0';
            }

            if (gpio_val == '0') {
                stuck_counter++;
                // Affiche un message d'alerte sans spammer le
                // terminal
                if (stuck_counter % 500000 == 0) {
                    printf("ATTENTION: La Jetson est bloquee, le
                        STM32 dit STOP (LOW)\n");
                }
            }
        } while (gpio_val == '0');

        // On envoie sur le SPI
        if (ioctl(spi_fd, SPI_IOC_MESSAGE(1), &tr) < 1) {
            perror("Erreur SPI");
            break;
        }
    }

    pclose(pipe);
    close(spi_fd);
    close(gpio_fd);
    return 0;
}

```

## D.2 RPI4B (rpi4b\_to\_rpi.c)

```

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <linux/spi/spidev.h>
#include <sys/mman.h>

```

```
#define CHUNK_SIZE 1400
#define SPI_DEVICE "/dev/spidev0.0"
#define GPIO_PIN 24

volatile unsigned *gpio;

// Initialisation du GPIO 24
void setup_gpiomem() {
    int mem_fd = open("/dev/gpiomem", O_RDWR | O_SYNC);
    if (mem_fd < 0) { perror("Erreur_gpiomem"); exit(1); }
    gpio = (volatile unsigned *)mmap(NULL, 4096, PROT_READ |
        PROT_WRITE, MAP_SHARED, mem_fd, 0);
    close(mem_fd);
    *(gpio + 2) &= ~(7 << 12); // GPIO 24 en entree
}

int main() {
    int spi_fd;
    uint32_t speed = 5000000; // 5 MHz
    uint8_t bits = 8;
    uint32_t mode = 0;

    setup_gpiomem();

    spi_fd = open(SPI_DEVICE, O_RDWR);
    if (spi_fd < 0) { perror("SPI_open"); return 1; }

    ioctl(spi_fd, SPI_IOC_WR_MODE, &mode);
    ioctl(spi_fd, SPI_IOC_WR_BITS_PER_WORD, &bits);
    ioctl(spi_fd, SPI_IOC_WR_MAX_SPEED_HZ, &speed);

    // cette commande peut donc varier en fonction des essais
    // video
    const char *video_cmd = "rpikam-vid -t 0 --width 640 --height 480 --framerate 60 --bitrate 400000 --profile baseline --inline --intra 60 --flush -o - 2>/dev/null";

    FILE *pipe = popen(video_cmd, "r");
    if (!pipe) return 1;

    setvbuf(pipe, NULL, _IONBF, 0);

    uint8_t buffer[CHUNK_SIZE];

    struct spi_ioc_transfer tr = {
        .tx_buf = (uint64_t)(uintptr_t)buffer,
        .rx_buf = 0,
        .len = CHUNK_SIZE,
    };
}
```

```

        .speed_hz = speed,
        .bits_per_word = bits,
        .cs_change = 0,
    };

    printf(">>>Flux video actif\n");

    int drop_counter = 0;

    while (1) {
        size_t bytes_read = fread(buffer, 1, CHUNK_SIZE, pipe);
        if (bytes_read == 0) break;

        if (bytes_read < CHUNK_SIZE) {
            memset(buffer + bytes_read, 0, CHUNK_SIZE - bytes_read);
        }

        int timeout_counter = 0;
        int stm32_ready = 1;

        while ( (*(gpio + 13) & (1 << GPIO_PIN)) == 0 ) {
            timeout_counter++;
            // Si le STM32 met plus de temps, c'est qu'il sature.
            if (timeout_counter > 50000) {
                stm32_ready = 0;
                break;
            }
        }

        if (stm32_ready) {
            if (ioctl(spi_fd, SPI_IOC_MESSAGE(1), &tr) < 1) {
                perror("Erreur SPI");
                break;
            }
            drop_counter = 0;
        } else {
            // LE STM32 SATURE (Bitrate Spike)
            drop_counter++;
            if (drop_counter == 1 || drop_counter % 50 == 0) {
                printf("[ATTENTION] Pic de mouvement : STM32 sature
                    , paquet jete!\n");
            }
        }
    }

    pclose(pipe);

```



```
close(spi_fd);
return 0;
}
```

## E Firmware MCU (spi\_interface.c)

```
#include <string.h>
#include <endian.h>
#include "mmosal.h"
#include "mmwlan.h"
#include "mmconfig.h"

#include "mmipal.h"
#include "lwip/icmp.h"
#include "lwip/tcpip.h"
#include "lwip/udp.h"
#include "lwip/netif.h"

#include "mm_app_common.h"
#include "stm32u5xx_hal.h"

// --- SPI ---
#define SPI_PAYLOAD_SIZE 1400

SPI_HandleTypeDef hspi1;
uint8_t spi_rx_buffer[SPI_PAYLOAD_SIZE];
volatile bool spi_packet_received = false;
// -----
/* Application default configurations. */

/** Number of broadcast packet to transmit */
#define DEFAULT_BROADCAST_PACKET_COUNT 100
/** UDP port to bind too. */
#define DEFAULT_UDP_PORT 1337
/** Interval between successive packet transmission. */
#define DEFAULT_PACKET_INTERVAL_MS 100
/** Maximum length of broadcast tx packet payload */
#define BROADCAST_PACKET_MAX_TX_PAYLOAD_LEN 35
/** Format string to use for the tx packet payload */
#define BROADCAST_PACKET_TX_PAYLOAD_FMT "G'day␣World,␣packet␣no.␣%lu."
/* Default mode for the application */
#define DEFAULT_UDP_BROADCAST_MODE TX_MODE
/** Default ID used in the rx metadata. */
#define DEFAULT_UDP_BROADCAST_ID 0
```

```

/** Key used to identify received broadcast packets. */
#define MMBC_KEY 0x43424d4d

/** Enumeration of the various broadcast modes that can be used. */
enum udp_broadcast_mode
{
    /** Transmit mode. Application will transmit a set amount of
        broadcast packets. */
    TX_MODE,
    /** Receive mode. Application will listen for any broadcast
        packets and process any that start
        * with @ref MMBC_KEY */
    RX_MODE
};

/** UDP broadcast rx payload format. */
PACK_STRUCT_STRUCT struct udp_broadcast_rx_payload
{
    /** Key used to identify payload.*/
    uint32_t key;

    /** Flexible array member used to access color data for each ID
        . */
    struct
    {
        /** Red intensity. */
        uint8_t red;
        /** Green intensity. */
        uint8_t green;
        /** Blue intensity. */
        uint8_t blue;
    } data[];
};

/** Struct used in rx mode for storing state. */
struct udp_broadcast_rx_metadata
{
    /** The last time in milliseconds that a valid payload was
        received. */
    uint32_t last_rx_time_ms;
    /** ID of the device, used to retrieve data from the payload.
        */
    uint32_t id;
};

// --- Variables de Profilage DWT ---
volatile uint32_t dwt_start = 0;

```

```
volatile uint32_t dwt_end = 0;
volatile uint32_t mcu_cycles = 0;
extern uint32_t SystemCoreClock; // Fourni par le systeme (ici 160
    MHz)

/** Global data structure used in RX mode to record metadata. */
static struct udp_broadcast_rx_metadata rx_metadata = { 0 };

static volatile bool is_network_ready = false;

/* Callback pour savoir quand la connexion est prete*/
static void link_status_callback(const struct mmipal_link_status *
    link_status)
{
    if (link_status->link_state == MMIPAL_LINK_UP) {
        printf("\n>>>_CONNECTE_A_OPENWRT_<<<\n");
        is_network_ready = true;
    }
}

/**
 * Callback function to handle received data from the UDP pcb.
 *
 * @warning Be aware that @c addr might point into the pbuf @c p so
 *         freeing this pbuf can make
 *         @c addr invalid, too.
 *
 * @param arg    User supplied argument used to store a reference to
 *               the global rx_metadata struct.
 * @param pcb    The udp_pcb which received data
 * @param p      The packet buffer that was received
 * @param addr   The remote IP address from which the packet was
 *               received
 * @param port   The remote port from which the packet was received
 */
static void udp_raw_recv(void *arg,
                        struct udp_pcb *pcb,
                        struct pbuf *p,
                        const ip_addr_t *addr,
                        u16_t port)
{
    LWIP_UNUSED_ARG(pcb);
    LWIP_UNUSED_ARG(addr);
    LWIP_UNUSED_ARG(port);

    if (p == NULL)
    {

```

```

        return;
    }

    struct udp_broadcast_rx_metadata *metadata = (struct
        udp_broadcast_rx_metadata *)arg;
    struct udp_broadcast_rx_payload *payload = (struct
        udp_broadcast_rx_payload *)p->payload;
    uint32_t current_time_ms = mmosal_get_time_ms();

    /* This is the minimum length we need to prevent reading off
       the end of the payload. */
    uint32_t min_payload_len =
        sizeof(payload->key) + (sizeof(payload->data[0]) * (
            metadata->id + 1));

    if (p->len < min_payload_len)
    {
        printf("Payload length too short. Len: %u. Min len: %lu\n",
            p->len, min_payload_len);
        goto exit;
    }

    if (le32toh(payload->key) != MMBC_KEY)
    {
        printf("Invalid payload received.\n");
        goto exit;
    }

    printf("Valid payload received.\n"
        "Time since last: %lums\n"
        "Data recieved: 0x%02x%02x%02x\n\n",
        (current_time_ms - metadata->last_rx_time_ms),
        payload->data[metadata->id].red,
        payload->data[metadata->id].green,
        payload->data[metadata->id].blue);

    metadata->last_rx_time_ms = current_time_ms;

    mmhal_set_led(LED_RED, payload->data[metadata->id].red);
    mmhal_set_led(LED_GREEN, payload->data[metadata->id].green);
    mmhal_set_led(LED_BLUE, payload->data[metadata->id].blue);

exit:
    pbuf_free(p);
}

/**

```

```

* Set a receive callback for the UDP PCB. This callback will be
  called when receiving a datagram
* for the pcb.
*
* @param pcb UDP protocol control block to register the callback
  for
*/
static void udp_broadcast_rx_start(struct udp_pcb *pcb)
{
    mmconfig_read_uint32("udp_broadcast.id", &(rx_metadata.id));

    LOCK_TCPIP_CORE();
    udp_recv(pcb, udp_raw_recv, &rx_metadata);
    UNLOCK_TCPIP_CORE();
}

/**
* Broadcast a udp packet every @ref DEFAULT_PACKET_INTERVAL_MS
  until @ref
* DEFAULT_BROADCAST_PACKET_COUNT packets have been sent.
*
* @note If the parameters are set in the config store they will be
  used.
*
* @param pcb UDP protocol control block to use for transmission
*/
static void udp_broadcast_tx_start(struct udp_pcb *pcb)
{
    //err_t err;

    ip_set_option(pcb, SOF_BROADCAST);
    ip_addr_t dest_ip;
    IP4_ADDR(ip_2_ip4(&dest_ip), 192, 168, 12, 255);

    // --- ACTIVATION DU COMPTEUR DE CYCLES DWT ---
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;

    printf(">>>PONT_SPI-WIFI_ACTIVE!En attente du SPI...<<<\n");
    ;
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET);
    uint32_t packet_counter = 0;
    while (1)
    {
        if (spi_packet_received) {

```

```

        struct pbuf *p = pbuf_alloc(PBUF_TRANSPORT,
        SPI_PAYLOAD_SIZE, PBUF_REF);
        if (p != NULL) {
            p->payload = (void *)spi_rx_buffer;

            LOCK_TCPIP_CORE();
            udp_sendto(pcb, p, &dest_ip, 1337);
            UNLOCK_TCPIP_CORE();

            pbuf_free(p);
        }

        // --- ARRET DU CHRONO ---
        dwt_end = DWT->CYCCNT;
        mcu_cycles = dwt_end - dwt_start;

        // Affichage 1 fois tous les 100 paquets (pour ne
        pas bloquer le CPU avec l'UART)
        if ((packet_counter++ % 100) == 0) {

            uint32_t freq_mhz = SystemCoreClock / 1000000;
            uint32_t time_us = mcu_cycles / freq_mhz;

            printf("[Profilage] Temps de traitement MCU
            : %lu us (%lu cycles)\n", time_us,
            mcu_cycles);
        }

        spi_packet_received = false;

        // STM32 ecoute
        HAL_SPI_Receive_IT(&hspi1, spi_rx_buffer,
        SPI_PAYLOAD_SIZE);

        // Ensuite GO pour le SPI
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET)
        ;

    } else {
        mmosal_task_sleep(1);
    }
}

/**
 * Initialize the UDP protocol control block. Binds to @ref
 * DEFAULT_UDP_PORT
 *

```

```

* @note If the parameters are set in the config store they will be
    used.
*
* @return Reference to the pcb is successfully initialized else
    NULL
*/
static struct udp_pcb *init_udp_pcb(void)
{
    struct udp_pcb *pcb = NULL;
    LOCK_TCPIP_CORE();
    pcb = udp_new();
    if (pcb != NULL) {
        udp_bind(pcb, IP_ANY_TYPE, 1337);
    }
    UNLOCK_TCPIP_CORE();
    return pcb;
}

/**
* Get the mode from config store.
*
* @return translates the value of @c udp_broadcast.mode into a
    @ref udp_broadcast_mode, if no valid
    mode is set @ref DEFAULT_UDP_BROADCAST_MODE is returned.
*/
static enum udp_broadcast_mode get_mode(void)
{
    enum udp_broadcast_mode mode = DEFAULT_UDP_BROADCAST_MODE;
    char mode_str[32];
    if (mmconfig_read_string("udp_broadcast.mode", mode_str, sizeof
        (mode_str)) > 0)
    {
        if (strcasecmp(mode_str, "tx") == 0)
        {
            mode = TX_MODE;
        }
        else if (strcasecmp(mode_str, "rx") == 0)
        {
            mode = RX_MODE;
        }
        else
        {
            printf("Unknown mode: %s. Reverting to default.\n",
                mode_str);
        }
    }

    return mode;
}

```

```
void SPI_Slave_Init(void)
{
    // Activer les horloges du SPI1, du Port E(SPI) et port D (
    // Spare GPIO)(go no go du TX spi)
    __HAL_RCC_SPI1_CLK_ENABLE();
    __HAL_RCC_GPIOE_CLK_ENABLE();
    __HAL_RCC_GPIOD_CLK_ENABLE();

    // D10(PE12), D13(PE13), D12(PE14) et D11(PE15)
    GPIO_InitTypeDef GPIO_InitStructure = {0};
    GPIO_InitStructure.Pin = GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 |
        GPIO_PIN_15;
    GPIO_InitStructure.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStructure.Pull = GPIO_NOPULL;
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_HIGH;

    GPIO_InitStructure.Alternate = GPIO_AF5_SPI1; // Sur U5, Port E =
        SPI1 (AF5) !
    HAL_GPIO_Init(GPIOE, &GPIO_InitStructure);

    // Config SPI1
    hspi1.Instance = SPI1;
    hspi1.Init.Mode = SPI_MODE_SLAVE;
    hspi1.Init.Direction = SPI_DIRECTION_2LINES;
    hspi1.Init.DataSize = SPI_DATASIZE_8BIT;
    hspi1.Init.CLKPolarity = SPI_POLARITY_LOW;
    hspi1.Init.CLKPhase = SPI_PHASE_1EDGE;

    // CS sur D10
    hspi1.Init.NSS = SPI_NSS_HARD_INPUT;

    hspi1.Init.FirstBit = SPI_FIRSTBIT_MSB;
    hspi1.Init.TIMode = SPI_TIMODE_DISABLED;
    hspi1.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLED;
    hspi1.Init.CRCPolynomial = 7;

    if (HAL_SPI_Init(&hspi1) != HAL_OK) {
        printf("ERREUR : Echec initialisation SPI1!\n");
    } else {
        printf("SPI Esclave (SPI1) initialise sur PE12 a PE15!\n");
    }
}

// --- LES 2 LIGNES MANQUANTES POUR LE MODE '_IT' --- mode IT ?
HAL_NVIC_SetPriority(SPI1_IRQn, 5, 0);
HAL_NVIC_EnableIRQ(SPI1_IRQn);
```



```

    // Initialisation de la broche Handshake (PD15) (Go no go
    spi)
    GPIO_InitTypeDef GPIO_InitStruct_Handshake = {0};
    GPIO_InitStruct_Handshake.Pin = GPIO_PIN_15;
    GPIO_InitStruct_Handshake.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct_Handshake.Pull = GPIO_NOPULL;
    GPIO_InitStruct_Handshake.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOD, &GPIO_InitStruct_Handshake);
}
// Quand le STM32 a reçu un paquet
void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef *hspi)
{
    if (hspi->Instance == SPI1) {
        // Demarrer le chrono
        dwt_start = DWT->CYCCNT;
        // mesure
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_RESET);
        spi_packet_received = true;
        //HAL_SPI_Receive_IT(&hspi1, spi_rx_buffer,
        SPI_PAYLOAD_SIZE); //ca faisait un double receive donc
        erreur -> donc un peut de lantence en plus
    }
}

void SPI1_IRQHandler(void)
{
    HAL_SPI_IRQHandler(&hspi1);
}

/**
 * Main entry point to the application. This will be invoked in a
 * thread once operating system
 * and hardware initialization has completed. It may return, but it
 * does not have to.
 */
void app_init(void)
{
    printf("\n\n---TX_SPI->STM32->WIFI---\n\n");

    SPI_Slave_Init();

    HAL_SPI_Receive_IT(&hspi1, spi_rx_buffer, SPI_PAYLOAD_SIZE);

    app_wlan_init();
    mmipal_set_link_status_callback(link_status_callback);

    printf("Connexion a l'AP OpenWrt en cours...\n");
}

```

```

app_wlan_start();

mmwlan_ate_override_rate_control(MMWLAN_MCS_2, MMWLAN_BW_2MHZ,
    MMWLAN_GI_NONE);
printf("forçage 2MHz/2MCS2 force.\n");
mmwlan_set_power_save_mode(MMWLAN_PS_DISABLED); // pour que
    quand il n'est pas sous load les ping passe bien

while (!is_network_ready) {
    mmosal_task_sleep(10);
}

struct udp_pcb *pcb = init_udp_pcb();
if (pcb != NULL) {
    udp_broadcast_tx_start(pcb);
}

(void) get_mode;
(void) udp_broadcast_rx_start;
}

```

---

## Glossaire

- Couche MAC** : (*Media Access Control*) Sous-couche du modèle réseau OSI chargée de contrôler l'accès physique au support de transmission (l'air, dans notre cas) et de gérer les acquittements.
- DWT** : (*Data Watchpoint and Trace*) Composant matériel intégré aux processeurs ARM Cortex-M permettant, entre autres, de compter les cycles d'horloge du cœur pour un profilage temporel avec une précision de l'ordre de la nanoseconde.
- EKH05 / EKH19** : Cartes d'évaluation (*Evaluation Kits*) développées par Morse Micro intégrant les puces Wi-Fi HaLow MM8108. L'EKH05 dispose d'entrées/sorties matérielles étendues (bus SPI, GPIO) pour l'IoT, tandis que l'EKH19 est conçue pour un interfaçage réseau standard (USB ou routeur).
- FreeRTOS** : Système d'exploitation temps réel (RTOS) open-source pour microcontrôleurs, garantissant l'exécution de tâches avec des délais de traitement déterministes.
- GPIO** : (*General Purpose Input/Output*) Broche physique d'un microcontrôleur pouvant être configurée par logiciel comme entrée ou sortie numérique (utilisée dans ce projet pour le signal de *Handshake*).
- GStreamer** : Framework multimédia open-source reposant sur une architecture de graphes (pipelines). Il permet de chaîner des éléments logiciels pour capturer, encoder, transmettre et décoder de la vidéo avec une latence minimale.
- Jetson Nano** : Nano-ordinateur développé par Nvidia, équipé d'un processeur ARM et d'un processeur graphique (GPU) intégré, particulièrement optimisé pour l'accélération matérielle de l'encodage vidéo (NVENC).
- LDPC** : (*Low-Density Parity-Check*) Algorithme mathématique très performant de correction d'erreurs (FEC), permettant de reconstruire des paquets radio corrompus sans avoir besoin de les retransmettre.
- LwIP** : (*Lightweight IP*) Pile réseau TCP/IP open-source, hautement optimisée pour les systèmes embarqués (microcontrôleurs) disposant de très peu de mémoire vive (RAM).
- MM8108** : Puce SoC (*System on a Chip*) développée par Morse Micro. C'est le composant électronique au cœur de ce projet, chargé de la modulation et de l'émission des ondes Wi-Fi HaLow.
- OpenHD** : Système d'exploitation open-source basé sur Linux, conçu par la communauté FPV pour transmettre de la vidéo numérique longue portée en détournant des puces Wi-Fi standards (2.4 et 5.8 GHz).
- OpenOCD** : (*Open On-Chip Debugger*) Logiciel libre agissant comme un serveur de débogage. Il permet de programmer et d'inspecter la mémoire d'un microcontrôleur depuis un PC via une interface matérielle.
- OpenWrt** : Distribution Linux open-source extrêmement légère, conçue pour remplacer les microprogrammes (*firmwares*) des routeurs réseau, offrant un contrôle bas niveau sur les interfaces de routage.

**Raspberry Pi** : Gamme de nano-ordinateurs monocartes très populaires tournant sous Linux. La version *Compute Module 4 (CM4)* est une déclinaison miniaturisée sans connecteurs, destinée à l'intégration dans des systèmes industriels ou embarqués.

**SPI** : (*Serial Peripheral Interface*) Bus de communication matériel synchrone fonctionnant en mode Maître/Esclave. Utilisé ici pour le transfert à très haute vitesse du flux vidéo compressé entre Linux et le microcontrôleur.

**ST-LINK** : Sonde de débogage matérielle développée par STMicroelectronics, permettant à un ordinateur de flasher du code et de contrôler un microcontrôleur STM32.

**STM32** : Famille de microcontrôleurs 32 bits basés sur l'architecture ARM Cortex-M, développée par STMicroelectronics.

**STM32CubeIDE** : Environnement de développement intégré (IDE) officiel de STMicroelectronics, basé sur Eclipse, utilisé pour écrire, compiler et déboguer le code C du projet embarqué.

**Tcpdump** : Outil d'analyse réseau en ligne de commande permettant de capturer et d'inspecter les paquets de données (trames) transitant sur une interface réseau matérielle (Wi-Fi ou Ethernet).

# Index

Couche MAC, 26

DWT, 35

EKH19, 21

FPV, 20

FreeRTOS, 18, 23

GPIO, 33

GStreamer, 21, 30, 35

Jetson Nano, 20

LDPC, 18

LwIP, 21, 23, 35

OpenHD, 36

OpenOCD, 24

OpenWrt, 18, 21

Raspberry Pi, 21

SPI, 21, 35

ST-LINK, 24

STM32, 18, 21

STM32CubeIDE, 23

Tcpdump, 32

Wi-Fi HaLow

MM8108, 18, 21